AWS Developer Certification

Jacopo Pela

October 29, 2024

Contents

1 Amazon SQS

Amazon SQS is a message queue service used by distributed applications to exchange messages through a **polling model**, and can be used to decouple sending and receiving components.

Amazon SQS and SNS are lightweight, fully managed message queue and topic services that scale almost infinitely and provide simple, easy-to-use APIs.

Amazon SQS provides queue with **FIFO** (first-in-first-out) strategy, queues preserve the exact order in which messages are sent and received.
Standard queues provide a **loose-FIFO** capability that attempts to preserve the order of messages, receiving messages in the exact order they are sent is not guaranteed.

Standard queues provide at-least-once delivery, which means that each message is delivered at least once.
FIFO queues provide exactly-once processing , which means that each message is delivered once and remains available until a consumer processes it and deletes it. Duplicates are not introduced into the queue.

Amazon SQS provides several advantages over building your own software for managing message queues or using commercial or open-source message queuing systems that require significant upfront time for development and configuration.

Alternatives require ongoing hardware maintenance and system administration resources.
The complexity of configuring and managing these systems is compounded by the need for redundant storage of messages that ensures messages are not lost if hardware fails.

Amazon SQS requires no administrative overhead and little configuration.
Amazon SQS works on a massive scale, processing billions of messages per day.
Amazon SQS also provides extremely high message durability
Amazon SQS offers a reliable, highly-scalable hosted queue for storing

messages as they travel between applications or microservices. It moves data between distributed application components and helps you decouple these components

1.1 Billing

Amazon SQS use the **on-demand** payment method. You pay only for what you use, and there is no minimum fee.
The cost of Amazon SQS is calculated per request, plus data transfer charges for data transferred out of Amazon SQS.

The Amazon SQS Free Tier provides you with **1 million requests per month** at no charge.
The Free Tier is a monthly offer. Free usage does not accumulate across months.
For any requests beyond the free tier. All Amazon SQS requests are chargeable, and they are billed at the same rate.
Batch operations all cost the same as other Amazon SQS requests. By grouping messages into batches, you can reduce your Amazon SQS costs.

You can tag and track your queues for resource and cost management using cost allocation tags.

1.2 Features, functionality, and interfaces

You can make your applications more flexible and scalable by using Amazon SQS with compute services as well as with storage and database services.

You can access Amazon SQS using the AWS Management Console.
Amazon SQS also provides a web services API. It is also integrated with the AWS SDKs, allowing you to work in the programming language of your choice.

Only an AWS account owner (or an AWS account that the account owner has delegated rights to) can perform operations on an Amazon SQS message queue.

You can take advantage of the scale, low cost, and high availability of Amazon SQS without the worry and high overhead of running your own **JMS** (Java Message Service) cluster.
Amazon provides the Amazon SQS Java Messaging Library that implements the JMS 1.1 specification and uses Amazon SQS as the **JMS provider**.

In Amazon SQS, you can configure a **dead letter queues**, which receive messages from other source queues. When configuring it you are required to set appropriate permissions for the dead letter queue redrive using **RedriveAllowPolicy**.
RedriveAllowPolicy includes the parameters for the dead-letter queue redrive permission. It defines which source queues can specify dead-letter queues as a JSON object.
Dead letter queue receives messages after a maximum number of processing attempts cannot be completed. You can use dead letter queues to isolate messages that can't be processed for later analysis.

Visibility timeout is a period of time during which Amazon SQS prevents other consuming components from receiving and processing a message.

An Amazon SQS message can contain **up to 10 metadata** attributes. You can use message attributes to separate the body of a message from the metadata that describes it.
This helps process and store information with greater speed and efficiency.
Amazon SQS message attributes take the form of name-type-value triples. The supported types include string, binary, and number (including integer, floating-point, and double).

To determine the **time-in-queue** value, you can request the **SentTimestamp** attribute when receiving a message. Subtracting that value from the current time results in the time-in-queue value.

Typical latencies for **SendMessage**, **ReceiveMessage**, and **DeleteMessage** API requests are in the tens or low hundreds of milliseconds.

For anonymous access the value of the **SenderId** attribute for a message is the **IP address**, when the AWS account ID is not available (for example, when an anonymous user sends a message).

Amazon SQS **long polling** is a way to retrieve messages from your Amazon SQS queues. While the regular short polling returns immediately, even if the message queue being polled is empty, long polling doesn't return a response until a message arrives in the message queue, or the long poll times out.
Long polling makes it inexpensive to retrieve messages from your Amazon SQS queue as soon as the messages are available. Using long polling might reduce the cost of using SQS, because you can reduce the number of empty receives.

Long-polling **ReceiveMessage** calls are billed exactly the same as short-polling ReceiveMessage calls.

Amazon SQS long polling is preferable to short polling. Long-polling requests let your queue consumers receive messages as soon as they arrive in your queue.
Amazon SQS long polling results in higher performance at reduced cost in the majority of use cases.
If your application expects an immediate response from a ReceiveMessage call, you might not be able to take advantage of long polling.

Use a maximum of **20 seconds** for a long-poll timeout. Higher long-poll timeout values reduce the number of empty **ReceiveMessageResponse** instances returned, try to set your long-poll timeout as high as possible.
If the 20-second maximum doesn't work for your application set a shorter long-poll timeout, as low as 1 second.

The **AmazonSQSBufferedAsyncClient** for Java provides an implementation of the **AmazonSQSAsyncClient** interface and adds several important features:

- Automatic batching of multiple SendMessage, DeleteMessage, or ChangeMessageVisibility requests without any required changes to the application

- Prefetching of messages into a local buffer that allows your application to immediately process messages from Amazon SQS without waiting for the messages to be retrieved

The AmazonSQSBufferedAsyncClient for Java is implemented as a drop-in replacement for the existing AmazonSQSAsyncClient.
If you update your application to use the latest AWS SDK and change your client to use the AmazonSQSBufferedAsyncClient for Java instead of the AmazonSQSAsyncClient.

You can delete all messages in an Amazon SQS message queue using the **PurgeQueue** action.
When you purge a message queue, all the messages previously sent to the message queue are deleted.
To delete only specific messages, use the DeleteMessage or **DeleteMessage-Batch** actions.

1.3 FIFO queues

FIFO queues are available in all AWS regions where Amazon SQS is available.
FIFO queues are designed to never introduce duplicate messages. However, your message producer might introduce duplicates.
Amazon SQS APIs provide **deduplication functionality** that prevents your message producer from sending duplicates. Any duplicates introduced by the message producer are removed within a **5-minute** deduplication interval.

You might occasionally receive a duplicate copy of a message. If you use a standard queue, you must design your applications to be idempotent.

Amazon SQS standard queues do not change to FIFO, you can still create standard queues.

You can not convert to a FIFO queue. You must choose the queue type when you create it. However, it is possible to move to a FIFO queue.

FIFO queues use the same API actions as standard queues, and the mechanics for receiving and deleting messages and changing the visibility timeout are the same. However, when sending messages, you must specify a message **group ID**.

Some AWS or external services that send notifications to Amazon SQS might not be compatible with FIFO queues, despite allowing you to set a FIFO queue as a target.

FIFO queues aren't currently compatible with the Amazon SQS Buffered Asynchronous Client.
FIFO queues are compatible with the Amazon SQS Extended Client Library for Java and the Amazon SQS Java Message Service (JMS) client.

FIFO queues support all metrics that standard queues support. For FIFO queues, all approximate metrics return accurate counts.

Messages are grouped into distinct, ordered "bundles" within a FIFO queue. For each message group ID, all messages are sent and received in strict order. Messages with different message group ID values might be sent and received out of order. You must associate a message group ID with a message. If you don't provide a message group ID, the action fails.
If multiple hosts send messages with the same message group ID are sent to a FIFO queue, Amazon SQS delivers the messages in the order in which they arrive for processing.

Amazon SQS support multiple producer. One or more producers can send messages to a FIFO queue. Messages are stored in the order that they were successfully received by Amazon SQS.
If multiple producers send messages in parallel, without waiting for the success response from SendMessage or SendMessageBatch actions, the order between producers might not be preserved.

Amazon SQS FIFO queues don't serve messages from the same message group to more than one consumer at a time.
If your FIFO queue has multiple message groups, you can take advantage of parallel consumers.

FIFO queues support up to **3,000 messages** per second with batching or up to **300 messages** per second without batching.
If you require higher throughput, you can enable high throughput mode for FIFO on the Amazon SQS console, which will support up to 70,000 messages per second without batching and even higher with batching.

The name of a FIFO queue must end with the **.fifo suffix**. The suffix counts towards the 80-character queue name limits.

1.4 Security and reliability

Amazon SQS stores all message queues and messages within a single, highly-available AWS region with multiple redundant Availability Zones (AZs).

Authentication mechanisms ensure that messages stored in Amazon SQS message queues are secured against unauthorized access. You can control who can send messages to a message queue and who can receive messages from a message queue. You can build your application to encrypt messages before they are placed in a message queue.
Amazon SQS has its own resource-based permissions system that uses policies written in the same language as IAM policies.
Amazon SQS supports the **HTTPS** and **TLS** protocols.
Amazon SQS supports versions 1.0, 1.1, and 1.2 of the TLS protocol in all regions.

ReceiveMessage and DeleteMessage are separate operations because when Amazon SQS returns a message to you, the message stays in the message queue whether or not you actually receive the message. You're responsible for deleting the message and the deletion request acknowledges that you're done processing the message.
If you don't delete the message, Amazon SQS will deliver it again when it receives another receive request.

FIFO queues never introduce duplicate for delete messages.
For standard queues, under rare circumstances, you might receive a previously-deleted message a second time.

When you issue a DeleteMessage request on a previously-deleted message, Amazon SQS returns a success response.

1.5 Server-Side encryption (SSE)

SSE lets you transmit sensitive data in encrypted queues. SSE protects the contents of messages in Amazon SQS queues using keys managed in the **AWS KMS**.
SSE encrypts messages as soon as Amazon SQS receives them. The messages are stored in encrypted form and Amazon SQS decrypts messages only when they are sent to an authorized consumer.

You can use SNS, Cloud Watch Events and S3 Events with encrypted queues.To do this you need to enable compatibility between AWS services and Queues with SSE.

Server-side encryption (SSE) for Amazon SQS is available in all AWS regions where Amazon SQS is available.

To enable SSE for a new or existing queue using the Amazon SQS API, specify the customer master key (CMK) ID: the **alias**, **alias ARN**, **key ID or key ARN** of an AWS-managed CMK or a **custom CMK** by setting the **KmsMasterKeyId** attribute of the **CreateQueue** or **SetQueueAttributes** action.

Amazon SQS queue support SSE on both standard and FIFO queues.

Before you can use SSE, you must configure AWS KMS key policies to allow encryption of queues and encryption and decryption of messages.
To enable SSE for a queue, you can use the **AWS-managed CMK** for Amazon SQS or a **custom CMK**.
To send messages to an encrypted queue, the producer must have the **kms:GenerateDataKey** and **kms:Decrypt** permissions for the CMK.
To receive messages from an encrypted queue, the consumer must have the **kms:Decrypt** permission for any CMK that is used to encrypt the messages in the specified queue. If the queue acts as a dead letter queue , the consumer must also have the **kms:Decrypt** permission for any CMK that is used to encrypt the messages in the source queue.

There are no additional Amazon SQS charges for using SSE with Amazon SQS. However, there are charges for calls from Amazon SQS to AWS KMS.

SSE encrypts the body of a message in an Amazon SQS queue.
SSE doesn't encrypt the following components:

- Queue metadata (queue name and attributes)
- Message metadata (message ID, timestamp, and attributes)
- Per-queue metrics

SSE uses the **AES-GCM 256 algorithm** for Amazon SQS use to encrypt messages.

SSE doesn't limit the throughput (TPS) of Amazon SQS. The number of SSE queues that you can create is limited by the following:

- The data key reuse period (1 minute to 24 hours).
- The AWS KMS per-account quota (100 TPS by default).
- The number of IAM users or accounts that access queues.
- The existence of a large backlog (a larger backlog requires more AWS KMS calls).

To predict costs and better understand your AWS bill, you might want to know how often Amazon SQS uses your CMK.
To calculate the number of API requests per queue (R), use the following formula:
R = B / D * (2 * P + C)
Where:

- B is the billing period (in seconds)
- D is the data key reuse period (in seconds)
- P is the number of producing principals that send to the Amazon SQS queue.
- C is the number of consuming principals that receive from the Amazon SQS queue.

If the producer and consumer have different IAM users, the cost increases.

1.6 Compliance

Amazon SQS is PCI DSS Level 1 certified.
AWS has expanded its **HIPAA compliance program** to include Amazon SQS as a **HIPAA Eligible Service**.
If you have an executed **Business Associate Agreement** (BAA) with AWS, you can use Amazon SQS to build your HIPAA-compliant applications, store messages in transit, and transmit messages—including messages containing protected health information (PHI).
If you already have an executed BAA with AWS, you can start using Amazon SQS right away.

1.7 Limits and restrictions

Longer message retention provides greater flexibility to allow for longer intervals between message production and consumption.
You can configure the Amazon SQS message retention period to a value from 1 minute to 14 days. The default is 4 days. Once the message retention quota is reached, your messages are automatically deleted.

To configure the message retention period, set the MessageRetentionPeriod attribute using the console or using the Distributiveness method. Use this attribute to specify the number of seconds a message will be retained in Amazon SQS.

To configure the maximum message size, use the console or the SetQueueAttributes method to set the MaximumMessageSize attribute. This attribute specifies the number of bytes that an Amazon SQS message can contain. Set this attribute to a value between 1 KB and 256 KB.

To send messages larger than 256 KB (up to 2GB), use the Amazon SQS Extended Client Library for Java.

Amazon SQS messages can contain up to 256 KB of text data, including XML, JSON and unformatted text.

A single Amazon SQS message queue can contain an unlimited number

of messages. However, there is a quota of 120,000 for the number of inflight messages for a standard queue and 20,000 for a FIFO queue.

You can create any number of message queues.

Queue names are limited to 80 characters.
You can use alphanumeric characters, hyphens (-), and underscores (_) on queue names.
A message queue's name must be unique within an AWS account and region.
You can reuse a message queue's name after you delete the message queue.

1.8 Queue sharing

You can associate an access policy statement (and specify the permissions granted) with the message queue to be shared.
The message queue owner pays for shared message queue access.

The Amazon SQS API uses the AWS account number to identify AWS users.
To share a message queue with an AWS user, provide the full URL from the message queue you want to share.
The **CreateQueue** and **ListQueues** operations return this URL in their responses.

You can configure an access policy that allows anonymous users to access a message queue.

The permissions API provides an interface for sharing access to a message queue to developers.

The SetQueueAttributes operation supports the full access policy language and operation with JSON object.

1.9 Service access and regions

Each Amazon SQS message queue is independent within each region.
Amazon SQS pricing is the same for all regions, except China (Beijing).
You can transfer data between Amazon SQS and Amazon EC2 or AWS Lambda free of charge within a single region.
When you transfer data between Amazon SQS and Amazon EC2 or AWS Lambda in different regions, you will be charged the normal data transfer rate.

1.10 Dead-letter queues

A **dead-letter queue** is an Amazon SQS queue to which a source queue can send messages if the source queue's consumer application is unable to consume the messages successfully. Dead-letter queues make it easier for you to handle message consumption failures and manage the life cycle of unconsumed messages.
Once you recover your consumer application, you can redrive the messages from your dead-letter queue to the source queue.

When you create your source queue, Amazon SQS allows you to specify a dead-letter queue (DLQ) and the condition under which SQS should move messages to the DLQ.
The condition is the number of times a consumer can receive a message from the queue, defined as maxReceiveCount. This is known as the redrive policy.
When the **ReceiveCount** for a message exceeds the **maxReceiveCount** for a queue, Amazon SQS is designed to move the message to a dead-letter queue.
The redrive policy manages the first half of the life cycle of unconsumed messages by moving them from a source queue to a dead-letter queue.

Dead-letter queue redrive to source queue work by allowing you to investigate a sample of messages available in the dead-letter queue by showing message attributes and related metadata.
Once you have investigated the messages, you can move them back to their source queue(s). You can also select the redrive velocity to configure the rate at which Amazon SQS will move the messages from the dead-letter queue to

the source queue.

You must use a FIFO dead letter queue with a FIFO queue.

AWS SQS has no limit on the messages stored queue

2 Amazon DynamoDB

DynamoDB is a **fast** and **flexible nonrelational database service** for any scale. DynamoDB enables customers to offload the administrative burdens of operating and scaling distributed databases to AWS so that they don't have to worry about hardware provisioning, setup and configuration, throughput capacity planning, replication, software patching, or cluster scaling.

2.1 About Amazon DynamoDB

DynamoDB takes away one of the main stumbling blocks of scaling databases: the management of database software and the provisioning of the hardware needed to run it.
DynamoDB automatically scales throughput capacity to meet workload demands, and partitions and repartitions your data as your table size grows.
DynamoDB synchronously replicates data across three facilities in an AWS Region, giving you high availability and data durability.

2.1.1 What is the consistency model of DynamoDB?

When reading data from DynamoDB, users can specify whether they want the read to be eventually consistent or strongly consistent:

- Eventually consistent reads (the default) – The eventual consistency option maximizes your read throughput. However, an eventually consistent read might not reflect the results of a recently completed write. All copies of data usually reach consistency within a second. Repeating a read after a short time should return the updated data.

- Strongly consistent reads — In addition to eventual consistency, DynamoDB also gives you the flexibility and control to request a strongly

consistent read if your application, or an element of your application, requires it. A strongly consistent read returns a result that reflects all writes that received a successful response before the read.

- ACID transactions – DynamoDB transactions provide developers atomicity, consistency, isolation, and durability (ACID) across one or more tables within a single AWS account and region. You can use transactions when building applications that require coordinated inserts, deletes, or updates to multiple items as part of a single logical business operation.

2.2 Getting started

DynamoDB supports GET/PUT operations by using a user-defined primary key. The primary key is the only required attribute for items in a table. You specify the primary key when you create a table, and it uniquely identifies each item. DynamoDB also provides flexible querying by letting you query on nonprimary key attributes using **global secondary indexes** and **local secondary indexes**.

A primary key can be either a single-attribute partition key or a composite partition-sort key.

DynamoDB indexes a composite partition-sort key as a partition key element and a sort key element. This multipart key maintains a hierarchy between the first and second element values.

After you have created a table using the DynamoDB console or CreateTable API, you can use the PutItem or BatchWriteItem APIs to insert items. Then, you can use the GetItem, BatchGetItem, or, if composite primary keys are enabled and in use in your table, the Query API to retrieve the items you added to the table.

DynamoDB is a fully managed cloud service that you access via API. Applications running on any operating system can use DynamoDB.

2.3 Planning

Each DynamoDB table has provisioned read-throughput and write-throughput associated with it. You are billed by the hour for that throughput capacity. Note that you are charged by the hour for the throughput capacity, whether or not you are sending requests to your table.
DynamoDB charges for data storage as well as the standard internet data transfer fees.

Maximum throughput per DynamoDB table is practically unlimited.

The smallest provisioned throughput you can request is 1 write capacity unit and 1 read capacity unit for both auto scaling and manual throughput provisioning. Such provisioning falls within the free tier which allows for 25 units of write capacity and 25 units of read capacity. The free tier applies at the account level, not the table level.
The DynamoDB Standard-Infrequent Access (DynamoDB Standard-IA) table class is optimized for tables that store data that is accessed infrequently, where storage is the dominant cost. Each table is associated with a table class and each table class offers a different pricing for data storage as well as read and write requests. You can select the most cost-effective table class based on your table's storage requirements and data access patterns.

DynamoDB Standard-IA helps you reduce your DynamoDB total costs for tables that store infrequently accessed data such as applications' logs, old social media posts, e-commerce order history, and past gaming achievements. If storage is your dominant table cost— storage cost exceeds 50 percent of the cost of throughput consistently.

DynamoDB Standard-IA tables are no different than DynamoDB Standard tables in supporting all existing DynamoDB features including global tables, secondary indexes, on-demand backups, and point-in-time recovery (PITR). DynamoDB Standard-IA tables also have built-in integration with other AWS services in the same way as DynamoDB Standard tables.

2.4 How It Works

In DynamoDB, tables, items, and attributes are the core components that you work with. A **table** is a collection of **items**, and each item is a collection of **attributes**. DynamoDB uses primary keys to uniquely identify each item in a table and secondary indexes to provide more querying flexibility.
You can use DynamoDB Streams to capture data modification events in DynamoDB tables.

The following are the basic DynamoDB components:

- **Tables** - DynamoDB stores data in tables. A table is a collection of data.
- **Items** - Each table contains zero or more items. An item is a group of attributes that is uniquely identifiable among all of the other items.
- **Attributes** - Each item is composed of one or more attributes. An attribute is a fundamental data element, something that does not need to be broken down any further.

2.4.1 Primary key

When you create a table, in addition to the table name, you must specify the primary key of the table. The primary key uniquely identifies each item in the table, so that no two items can have the same key.

DynamoDB supports two different kinds of primary keys:

- **Partition key**: A simple primary key, composed of one attribute known as the partition key. DynamoDB uses the partition key's value as input to an internal hash function. The output from the hash function determines the partition (physical storage internal to DynamoDB) in which the item will be stored.
  In a table that has only a partition key, no two items can have the same partition key value.
- **Partition key and sort key**: Referred to as a composite primary key, this type of key is composed of two attributes. The first attribute is the partition key, and the second attribute is the sort key.

DynamoDB uses the partition key value as input to an internal hash function. The output from the hash function determines the partition (physical storage internal to DynamoDB) in which the item will be stored. All items with the same partition key value are stored together, in sorted order by sort key value.
In a table that has a partition key and a sort key, it's possible for multiple items to have the same partition key value. However, those items must have different sort key values.

Each primary key attribute must be a **scalar**. The only data types allowed for primary key attributes are string, number, or binary. There are no such restrictions for other, non-key attributes.

2.4.2 Secondary indexes

You can create one or more secondary indexes on a table. A secondary index lets you query the data in the table using an alternate key, in addition to queries against the primary key. DynamoDB doesn't require that you use indexes, but they give your applications more flexibility when querying your data.
DynamoDB supports two kinds of indexes:

- **Global secondary index**: An index with a partition key and sort key that can be different from those on the table.
- **Local secondary index**: An index that has the same partition key as the table, but a different sort key.

Each table in DynamoDB has a quota of 20 global secondary indexes (default quota) and 5 local secondary indexes.

2.4.3 DynamoDB Streams

DynamoDB Streams is an optional feature that captures data modification events in DynamoDB tables. The data about these events appear in the stream in near-real time, and in the order that the events occurred.

Each event is represented by a stream record. If you enable a stream on a table, DynamoDB Streams writes a stream record whenever one of the following events occurs:

- A new item is added to the table: The stream captures an image of the entire item, including all of its attributes.
- An item is updated: The stream captures the "before" and "after" image of any attributes that were modified in the item.
- An item is deleted from the table: The stream captures an image of the entire item before it was deleted.

Each stream record also contains the name of the table, the event timestamp, and other metadata. Stream records have a lifetime of 24 hours; after that, they are automatically removed from the stream.

You can use DynamoDB Streams together with AWS Lambda to create a trigger.
ynamoDB Streams enables powerful solutions such as data replication within and across AWS Regions, materialized views of data in DynamoDB tables, data analysis using Kinesis materialized views, and much more.

2.5 Scalability, availability, and durability

DynamoDB lets you offload the administrative burdens of operating and scaling a distributed database so that you don't have to worry about hardware provisioning, setup and configuration, replication, software patching, or cluster scaling. DynamoDB also offers **encryption at rest**, which eliminates the operational burden and complexity involved in protecting sensitive data.

You can **scale up** or **scale down** your tables' throughput capacity without downtime or performance degradation. You can use the **AWS Management Console** to monitor resource utilization and performance metrics.

DynamoDB provides **on-demand backup capability**. It allows you to create full backups of your tables for long-term retention and archival for regulatory compliance needs.

You can create on-demand backups and enable **point-in-time** recovery for your Amazon DynamoDB tables. Point-in-time recovery helps protect your tables from accidental write or delete operations. With point-in-time

recovery, you can restore a table to any point in time during the **last 35 days**.
DynamoDB allows you to delete expired items from tables automatically to help you reduce storage usage and the cost of storing data that is no longer relevant.

2.5.1 High availability and durability

DynamoDB automatically spreads the data and traffic for your tables over a sufficient number of servers to handle your throughput and storage requirements, while maintaining consistent and fast performance. All of your data is stored on solid-state disks (SSDs) and is automatically replicated across multiple Availability Zones in an AWS Region, providing built-in high availability and data durability. You can use global tables to keep DynamoDB tables in sync across AWS Regions.

You can use AWS Database Migration Service (AWS DMS) to migrate data from a relational database or MongoDB to a DynamoDB table.

2.6 Auto scaling

Many database workloads are cyclical in nature, while others are difficult to predict in advance.
Amazon DynamoDB auto scaling uses the AWS Application Auto Scaling service to dynamically adjust provisioned throughput capacity on your behalf, in response to actual traffic patterns. This enables a table or a global secondary index to increase its provisioned read and write capacity to handle sudden increases in traffic, without throttling.

With Application Auto Scaling, you create a scaling policy for a table or a global secondary index. The scaling policy specifies whether you want to scale read capacity or write capacity (or both), and the minimum and maximum provisioned capacity unit settings for the table or index.
The scaling policy also contains a target utilization—the percentage of consumed provisioned throughput at a point in time. Application Auto Scaling uses a target tracking algorithm to adjust the provisioned throughput of the

table (or index) upward or downward in response to actual workloads.

Auto scaling can be triggered when two data points breach the configured target utilization value within a one-minute span. Therefore, auto scaling can take place because the consumed capacity is above target utilization for two consistent minutes. But if the spikes are more than a minute apart, auto scaling might not be triggered. Similarly, a scale down event can be triggered when 15 consecutive data points are lower than the target utilization. In either case, after auto scaling is triggered an UpdateTable call is invoked. It can then take several minutes to update the provisioned capacity for the table or the index. During this period, any requests that exceed the previous provisioned capacity of the tables will be throttled.

Important

You can't adjust the number of data points to breach to trigger the underlying alarm (though the current number could change in the future).

You can set the auto scaling target utilization values between **20** and **90 percent** for your read and write capacity.

Important

In addition to tables, DynamoDB auto scaling also supports global secondary indexes. Every global secondary index has its own provisioned throughput capacity, separate from that of its base table. When you create a scaling policy for a global secondary index, Application Auto Scaling adjusts the provisioned throughput settings for the index to ensure that its actual utilization stays at or near your desired utilization ratio.

2.6.1 Usage notes

DynamoDB auto scaling doesn't prevent you from manually modifying provisioned throughput settings. These manual adjustments don't affect any existing CloudWatch alarms that are related to DynamoDB auto scaling.

DynamoDB can use **conditional write** ti avoid overwrite the DB with unuseful information

DynamoDB DAX is a local inline fully managed chace the it's aimed to solve the **hot key** problem

DynamoDB **UpdateItem** action automatically create the item if it's not available yet, like the **PutItem** action

DynamoDB Transitional Write is a "all-or-nothing" type of write operation in a DynamoDB

3 Amazon ElastiCache

Amazon ElastiCache is a web service that streamlines deployment and running of Memcached or Redis protocol-compliant caches in the cloud. ElastiCache improves application performance by allowing you to retrieve information from a fast, managed, in-memory system instead of relying on slower disk-based systems.
The service simplifies and offloads the management, monitoring, and operation of in-memory environments, enabling your engineering resources to focus on developing applications. Using ElastiCache, you can not only improve load and response times to user actions and queries but also reduce the cost associated with scaling web applications.

ElastiCache automates common administrative tasks required to operate a distributed in-memory key-value environment. Using ElastiCache, you can add a caching or in-memory layer to your application architecture in a matter of minutes with a few steps.

ElastiCache is designed to automatically maintain high availability, and provides a **99.99% availability Service Level Agreement** (SLA). ElastiCache is protocol compliant with **Memcached** and **Redis**, so code, applications, and popular tools that you use today with your existing Memcached or Redis environments will seamlessly work with the service. There are **no up-front investments** required, and you pay only for the resources you use.

The **in-memory caching** provided by ElastiCache can be used to significantly **improve latency and throughput** for many read-heavy application workloads.
In-memory caching improves application performance by storing critical pieces of data in memory for **low-latency access**.

ElastiCache manages the work involved in setting up a distributed in-memory environment.
There is no infrastructure that you need to configure and manage. When designing your own ElastiCache cluster, the service automates common administrative tasks such as software patching and failure detection and recovery. ElastiCache provides detailed monitoring metrics associated with your resources, allowing you to quickly diagnose and react to issues.

ElastiCache offers fully managed Redis and Memcached for your most demanding applications that require submillisecond response times.

3.1 Serverless

ElastiCache Serverless is a serverless option that lets you get started with a cache in under a minute without infrastructure provisioning or capacity planning. ElastiCache Serverless removes the need for time-consuming capacity planning by continuously monitoring a cache's compute, memory, and network utilization, so it can instantly scale to meet demand without downtime or performance degradation.
ElastiCache Serverless automatically replicates data across multiple Availability Zones (AZs) and provides customers with a 99.99% availability SLA for each cache. With ElastiCache Serverless, you only pay for the data you store and the compute resources your application uses.

You can move an existing ElastiCache workload by changing the Redis or Memcached endpoint to your new ElastiCache Serverless cache endpoint in your application. You can migrate existing ElastiCache data to ElastiCache Serverless by specifying the Amazon Simple Storage Service (Amazon S3) location of a backup file.

ElastiCache Serverless supports ElastiCache for Redis version 7.1 and

ElastiCache for Memcached version 1.6.21 and above.

ElastiCache Serverless continuously monitors your cache's memory, compute, and network utilization to instantly scale. ElastiCache Serverless scales without downtime or performance degradation to the application by allowing the cache to scale up and initiating scale-out in parallel, to meet application requirements just in time.

With ElastiCache Serverless, you only pay for the data you store and the compute your application uses.

3.2 Reserved Nodes

Reserved Nodes or Reserved Instances (RIs) provide you with a significant discount over on-demand usage when you commit to a one-year or three-year term.
With Reserved Nodes, you can make a one-time, up-front payment to create a one- or three-year reservation to run your cache in a specific Region and receive a significant discount off the ongoing hourly usage charge.

There are three Reserved Node types: **All Upfront**, **No Upfront** and **Partial Upfront**.
Reserved Nodes provide a discount that applies to ElastiCache on-demand usage. ElastiCache Serverless is NOT compatible with Reserved Nodes.

You can purchase up to 300 Reserved Nodes. If you wish to run more than 300 nodes, please complete the ElastiCache node request form.

Purchase a node reservation with the same node class within the same Region as the node you are currently running and would like to reserve. If the reservation purchase is successful, ElastiCache will automatically apply your new hourly usage charge to your existing node.

Pricing changes associated with a Reserved Node are activated once your request is received and while the payment authorization is processed. You can follow the status of your reservation on the AWS Account Activity page

or by using the DescribeReservedCacheNodes API.

When your reservation term expires, your Reserved Node will revert to the appropriate On-Demand hourly usage rate for your node class and Region.

The ElastiCache APIs to create, modify, and delete nodes do not distinguish between On-Demand and Reserved Nodes, so that you can seamlessly use both.

Each Reserved Node is associated with a specific Region, which is fixed for the lifetime of the reservation and cannot be changed.

You cannot cancel your node reservation, and the one-time payment (if applicable) is not refundable.

When you purchase a Reserved Node under the All Upfront payment option, you pay for the entire term of the Reserved Node in one upfront payment. You can choose to pay nothing upfront by choosing the No Upfront option. The entire value of the No Upfront Reserved Node is spread across every hour in the term and you will be billed for every hour in the term, regardless of usage.
The Partial Upfront payment option is a hybrid of the All Upfront and No Upfront options.

3.2.1 Security

ElastiCache allows you to configure encryption of data at rest using **AWS KMS**, encryption of data in transit using **TLS**, authentication using **AWS IAM**, and network access control with **Amazon EC2** security groups.

When not using **Amazon VPC**, ElastiCache allows you to control access to your caches through network security groups. A security group acts like a firewall, controlling network access to your cache. By default, network access is turned off to your caches.

You can also control access to your ElastiCache resources using IAM authentication.

3.2.2 Compliance

ElastiCache supports compliance programs such as SOC 1, SOC 2, SOC 3, ISO, MTCS, C5, PCI DSS, HIPAA, and FedRAMP.
AWS PCI compliance program includes ElastiCache as a PCI-compliant service.

ElastiCache is a **HIPAA-eligible** service and is covered under the AWS Business Associate Addendum (BAA).
This means you can use ElastiCache to help you process, maintain, and store protected health information (PHI) and power healthcare applications.

If you have an executed Business Associate Agreement (BAA) with AWS, you can use ElastiCache to build applications that store and process HI under HIPAA.

The AWS FedRAMP compliance program includes ElastiCache Redis as a FedRAMP-authorized service. U.S. government customers and their partners can now use the latest version of ElastiCache to process and store their FedRAMP systems, data, and mission-critical, high-impact workloads in the AWS GovCloud (US-East) and AWS GovCloud (US-West) Regions,

There is no additional cost for using compliance features.

3.2.3 Parameter groups

A parameter group acts as a container for **engine configuration** values that can be applied to one or more clusters.
If you create a cluster without specifying a parameter group, a default parameter group is used. This default group contains engine defaults and ElastiCache system defaults optimized for the cluster you are running. However, if you want your cache to run with your custom-specified engine configuration values, you can create a new parameter group, modify the desired parame-

ters, and modify the cluster to use the new parameter group.
All clusters that use a particular parameter group get all the parameter updates to that parameter group.

ElastiCache by default chooses the optimal configuration parameters for your cache, taking into account the node type's memory and compute resource capacity. However, if you want to change them, you can do so using our **configuration management APIs**.

You can use the console, ElastiCache APIs, or command line tools to see information about your parameter groups and their corresponding parameter settings.

3.3 Redis

ElastiCache for Redis is a web service that streamlines deployment and running of Redis protocol-compliant caches in the cloud.
The service enables the management, monitoring, and operation of Redis nodes; creation, deletion, and modification of the nodes can be carried out through the ElastiCache console, the AWS CLI, or the web service APIs.
ElastiCache for Redis supports high-availability configurations, including Redis cluster mode enabled and cluster mode disabled with auto-failover from primary to replica.

ElastiCache for Redis is designed to be protocol compliant with open source Redis. Code, applications, drivers, and tools you use today with your existing standalone Redis data store will continue to work with ElastiCache for Redis and no code changes will be required for existing Redis deployments migrating to ElastiCache for Redis unless noted.

With ElastiCache for Redis you can create a read replica in another AWS AZ. When using ElastiCache Serverless for Redis, data is automatically stored redundantly across multiple AZs for high availability. When designing your own ElastiCache for Redis cache, upon failure of a node, we will provision a new node. In scenarios where the primary node fails, ElastiCache for Redis will automatically promote an existing read replica to the primary role.

You can quickly upgrade to a newer engine version by using the ElastiCache APIs and specifying your preferred engine version. In the ElastiCache console, you can select a cache and select Modify. The engine upgrade process is designed to retain your existing data.

Downgrading to an earlier engine version is not supported.

You can create **cross-Region** replicas using the **Global Datastore** feature in ElastiCache for Redis. Global Datastore provides fully managed, fast, reliable, and security-focused cross-Region replication. It allows you to write to your ElastiCache for Redis cluster in one Region and have the data available to be read from **up to two** other cross-Region replica clusters

3.3.1 Performance

ElastiCache for Redis provides enhanced I/O threads that deliver significant improvements to throughput and latency at scale through multiplexing, presentation layer offloading, and more. Enhanced I/O threads improve performance by using more cores for processing I/O and dynamically adjusting to the workload. ElastiCache for Redis improves the throughput of TLS-enabled clusters by offloading encryption to the same enhanced I/O threads. ElastiCache for Redis version 7.0 introduced enhanced I/O multiplexing, that combines many client requests into a single channel and improves the main Redis thread efficiency.

In ElastiCache for Redis version 7.1 and above, we extended the enhanced I/O threads functionality to also handle the presentation layer logic. Enhanced I/O threads not only read client input, but also parses the input into a Redis binary command format, which is then forwarded to the main thread for execution, to provide performance gains.
With ElastiCache for Redis version 7.1, you can achieve up to 100% more throughput and 50% lower P99 latency, compared to ElastiCache for Redis version 7.0. On r7g.4xlarge or larger, you can achieve over 1 million requests per second (RPS) per node.

ElastiCache provides two different sets of metrics to measure the CPU utilization of your cache depending on your cache deployment choice. When using ElastiCache Serverless, you can monitor the CPU utilization with the

ElastiCache Processing Units (ECPU) metric. The number of ECPUs consumed by your requests depends on the vCPU time taken and the amount of data transferred. Each read and write, like the Redis GET and SET commands or the Memcached get and set commands, requires 1 ECPU for each kilobyte (KB) of data transferred. Some Redis commands that operate on in-memory data structures can consume more vCPU time than a GET or SET command. ElastiCache calculates the number of ECPUs consumed based on the vCPU time taken by the command compared to a baseline of the vCPU time taken by a Redis SET or GET command. If your command takes additional vCPU time and transfers more data than the baseline of 1 ECPU, then ElastiCache calculates the ECPUs required based on the higher of the two dimensions.

When designing your own cluster, you can monitor EngineCPUUtilization and CPUUtilization. The CPUUtilization metric measures the CPU utilization for the instance (node), and the EngineCPUUtilization metric measures the utilization at the Redis process level. You need the EngineCPUUtilization metric in addition to the CPUUtilization metric, as the main Redis process is single threaded and uses just one CPU of the multiple CPU cores available on an instance. Therefore, the CPUUtilization metric does not provide precise visibility into the CPU utilization rates at the Redis process level. We recommend that you use both the CPUUtilization and EngineCPUUtilization metrics together to get a detailed understanding of CPU utilization for your Redis clusters.

Both sets of metrics are available in all AWS Regions, and you can access these metrics using Amazon CloudWatch.

Redis is one of the most popular NoSQL key-value stores and is known for its great performance.
In order to optimize the usage of Redis, you also need a performant client.

3.3.2 Read replica

Read replicas serve two purposes in Redis:

- Failure handling

- Read scaling

When you run a cache with a read replica, the primary serves both writes and reads. The replica serves exclusively read traffic and is also available as a warm standby in the event that the primary becomes impaired.

With ElastiCache Serverless, read replicas are automatically maintained by the service. When designing your own cache, there are a variety of scenarios where deploying one or more read replicas for a given primary node might make sense.
Common reasons for deploying a read replica include:

- Scaling beyond the compute or I/O capacity of a single primary node for read-heavy workloads: This excess read traffic can be directed to one or more read replicas.

- Serving read traffic while the primary is unavailable: If your primary node cannot take I/O requests you can direct read traffic to your read replicas.
  The read replica might be stale, since the primary instance is unavailable. The read replica can also be used warmed up to restart a failed primary.

- Data protection scenarios: In the unlikely event of primary node failure or that the AZ in which your primary node resides becomes unavailable, you can promote a read replica in a different AZ to become the new primary.

You can connect to a read replica just as you would connect to a **primary cache node**. If you have **multiple read replicas**, it is up to your application to determine how read traffic will be distributed amongst them. Here are some strategy:

- Redis clusters use the individual node endpoints for read operations.

- Redis clusters use the cluster's configuration endpoint for all operations. You must use a client that supports **Redis Cluster 3.2**. You can still read from individual node endpoints.

ElastiCache allows you to create up to **five read replicas**for a given primary cache node.

In the event of a failover, any associated and available read replicas should automatically resume replication once failover has completed.

Updates to a primary cache node will automatically be replicated to any associated read replicas. However, with **Redis' asynchronous replication technology**, a read replica can fall behind its primary cache node for a variety of reasons.
Typical reasons include:

- Write I/O volume to the primary cache node exceeds the rate at which changes can be applied to the read replica.
- Network partitions or latency between the primary cache node and a read replica.

Read replicas are subject to the strengths and weaknesses of Redis replication. If you are using read replicas, you should be aware of the potential for **lag between a read replicaand its primary cache node** or "inconsistency."

A read replica is billed as a standard cache node and at the same rates. Just like a standard cache node, the rate per cache node hour for a read replica is determined by the cache node class of the read replica.
You are not charged for the data transfer incurred in replicating data between your primary cache node and read replica. Billing for a read replica begins as soon as the read replica has been successfully created.
The read replica will continue being billed at standard ElastiCache cache node hour rates until you issue a command to delete it.

Initiated failover is supported by ElastiCache so that you can resume cache operations as quickly as possible. When failing over, ElastiCache flips the DNS record for your cache node to point at the read replica, which is in turn promoted to become the new primary. We encourage you to follow best practices and implement cache node connection retry at the application layer. Typically, start to finish, steps one to five below complete within six minutes.

These are the automatic failover events, listed in order of occurrence:

- Replication group message: Test Failover API called for node group <node-group-id>
- Cache cluster message: Failover from primary node <primary-node-id> to replica node <node-id> completed
- Replication group message: Failover from primary node <primary-node-id> to replica node <node-id> completed
- Cache cluster message: Recovering cache nodes <node-id>
- Cache cluster message: Finished recovery for cache nodes <node-id>

Your read replica can only be provisioned in the **same or different AZ** of the same Region as your cache node primary. You can, however, use the **Global Datastore for Redis** to work with fully managed, fast, reliable, and security-focused replication **across AWS Regions**. Using this feature, you can create cross-Region read replica clusters for **ElastiCache for Redis** to enable low-latency reads and disaster recovery across AWS Regions.

You can gain visibility into the location of the current primary by using the console or **DescribeCacheClusters API**.

You can add or remove a read replica across one or more shards in a Redis Cluster environment. The cluster continues to stay online and serve incoming I/O during this operation.

3.3.3 Multi-AZ

Multi-AZ is a feature that allows you to run in a more highly available configuration when designing your own ElastiCache for Redis cache. All ElastiCache Serverless caches are automatically run in a Multi-AZ configuration. An ElastiCache for Redis replication group consists of a primary and up to five read replicas. If Multi-AZ is enabled, then at least one replica is required per primary.
During certain types of planned maintenance, or in the unlikely event of an ElastiCache node failure or AZ failure, ElastiCache will automatically detect

the failure of a primary, select a read replica, and promote it to become the new primary.
ElastiCache also propagates the DNS changes of the promoted read replica, so if your application is writing to the primary node endpoint, no endpoint change will be needed.

The main benefits of running your ElastiCache for Redis in Multi-AZ mode are **enhanced availability**and a **smaller need for administration**.
When running ElastiCache in a Multi-AZ configuration, your caches are eligible for the **99.99% availability SLA**.
If an ElastiCache for Redis primary node failure occurs, the impact on your ability to read and write to the primary is limited to the time it takes for automatic failover to complete.
When Multi-AZ is enabled, ElastiCache node failover is automatic and requires no administration. You no longer need to monitor your Redis nodes and manually initiate a recovery in the event of a primary node disruption.

You can use Multi-AZ if you are using ElastiCache for Redis and have a replication group consisting of a primary node and one or more read replicas. If the primary node fails, ElastiCache will automatically detect the failure, select one from the **available read replicas**, and promote it to become the **new primary**.
ElastiCache will propagate the DNS changes of the promoted replica so that your application can keep writing to the primary endpoint. ElastiCache will also spin up a new node to replace the promoted read replica in the same AZ of the failed primary. In case the primary failed due to temporary AZ disruption, the new replica will be launched once that AZ has recovered.

Note that placing both the primary and the replicas in the **same AZ** will not make your ElastiCache for Redis replication group **resilient to an AZ disruption**. Additionally, having replicas in the same AZ as the primary will not be allowed if **Multi-AZ is turned on**.

ElastiCache will fail over to a read replica in the event of any of the following:

- Loss of availability in primary's AZ

- Loss of network connectivity to primary
- Compute unit failure on primary

If there are more than one read replicas, the read replica with the **smaller asynchronous replication lag to the primary** will be promoted.

ElastiCache will create an event to inform you that automatic failover occurred. You can use the **DescribeEvents API** to return information about events related to your ElastiCache node, or select the Events section in the ElastiCache Management Console.

AZs are engineered to provide low latency network connectivity to other AZs in the same Region. You should consider architecting your application and other AWS resources with redundancy across multiple AZs so your application will be resilient in the event of an AZ disruption.

3.3.4 Backup and restore

Backup and Restore is a feature that allows you to create snapshots of your ElastiCache for Redis caches. ElastiCache stores the snapshots, allowing users to subsequently use them to **restore Redis caches**.

Creating snapshots can be useful in case of data loss caused by node failure, as well as the unlikely event of a hardware failure.
Snapshots are stored in **Amazon S3**.

When a backup is initiated, ElastiCache will take a snapshot of a specified Redis cache that can later be used for recovery or archiving. You can initiate a backup any time you choose or set a recurring daily backup with retention period of up to 35 days. When you choose a snapshot to restore, a new ElastiCache for Redis cache will be created and populated with the snapshot's data.
ElastiCache for Redis snapshots are compatible with the open source **Redis RDB file format**.

You can use the Backup and Restore feature through the console, ElastiCache APIs, and AWS CLI.

Backup and Restore creates snapshots on a per-cache basis. Users can specify which ElastiCache for Redis cache to back up through the console, AWS CLI, or the ElastiCache API.
When using **ElastiCache Serverless**, backups are automatically conducted against read replicas.

You can export your ElastiCache for Redis snapshots to an authorized S3 bucket in the same Region as your cache.
You must first copy your snapshot into an authorized S3 bucket of your choice in the same Region and then grant cross-account bucket permissions to the other account.

ElastiCache provides storage space for one snapshot free of charge for each active ElastiCache for Redis cache. Additional storage will be charged based on the space used by the snapshots with $0.085/GB every month.
Data transfer for using the snapshots is free of charge.

When you delete an ElastiCache for Redis cache, your manual snapshots are retained. You will also have an option to create a final snapshot before the cache is deleted.
Automatic cache snapshots **are not retained**.

3.3.5 Enhanced engine

The engine within ElastiCache for Redis is fully compatible with open source Redis but also comes with enhancements that improve **performance, robustness, and stability**.
Some of the enhancements include:

- **More usable memory**: You can now safely allocate more memory for your application without risking increased swap usage during syncs and snapshots

- **Improved synchronization**: More robust synchronization under heavy load and when recovering from network disconnections. Additionally, syncs are faster as both the primary and replicas no longer use the disk for this operation.

- **Smoother failovers**: In the event of a failover, your shard now recovers faster as replicas no longer flush their data to do a full resync with the primary
- **TLS offload and IO multiplexing**: ElastiCache is designed to better use available CPU resources by handling certain network-related processes on dedicated threads.

The enhanced engine is **fully compatible with open source Redis**, thus you can enjoy its improved robustness and stability without the need to make any changes to your application code.

There is no additional charge for using the enhanced engine.

3.3.6 Encryption

Encryption at rest provides mechanisms to guard against unauthorized access of your data. When enabled, it encrypts the following aspects:

- Disk during sync, backup, and swap operations
- Backups stored in Amazon S3

ElastiCache for Redis offers default (service-managed) encryption at rest, as well as the ability to use your own symmetric customer-managed **AWS KMS keys** in AWS KMS.

The encryption in transit feature facilitates encryption of communications between clients and ElastiCache for Redis, as well as between the Redis servers.

Encryption in transit, encryption at rest, Redis AUTH, and Role-Based Access Control (RBAC) are features you can select when creating your ElastiCache for Redis cache. If you enabled in-transit encryption, you can choose to use Redis AUTH or RBAC for additional security and access control.

ElastiCache manages certification expiration and renewal behind the scenes. No user action is necessary for ongoing certificate maintenance.

There are **no additional costs** for using encryption.

3.3.7 Global Datastore

Global Datastore is a feature of ElastiCache for Redis that provides fully managed, fast, reliable and security-focused cross-Region replication. With Global Datastore, you can write to your ElastiCache for Redis cache in one Region and have the data available for read in **up to two other cross-Region replica clusters**, thereby enabling low-latency reads and disaster recovery across Regions.

Designed for real-time applications with a global footprint, Global Datastore typically replicates data across Regions in under one second, increasing the responsiveness of your applications by providing geolocal reads closer to end users. In the unlikely event of Regional degradation, one of the healthy cross-Region replica caches can be promoted to become the primary with full read and write capabilities.
Once initiated, the promotion typically completes in less than a minute, allowing your applications to remain available.

You can replicate to up to two secondary Regions within a Global Datastore. The caches in secondary Regions can be used to serve low-latency local reads and for disaster recovery in the unlikely event of Regional degradation.

Global Datastore is supported on ElastiCache for Redis 5.0.6 onward.

You can set up a Global Datastore by using an existing cache or creating a new cache to be used as a primary. You can create a Global Datastore with just a few steps in the ElastiCache Management Console or by downloading the latest AWS SDK or AWS CLI.

ElastiCache doesn't automatically promote a secondary cluster in the event when a primary cluster (Region) is degraded. You can manually initiate the failover by promoting a secondary cluster to become a primary.
The fail over and promotion of a secondary cluster typically completes in less than one minute.

Global Datastore uses encryption in transit for cross-Region traffic to keep your data more secure. Additionally, you can also encrypt your primary and secondary caches using encryption at rest to keep your data better secured. Each primary and secondary cache can have a separate customer-managed AWS KMS key for encryption at rest.

ElastiCache doesn't provide an SLA for RPO and RTO. The RPO varies based on replication lag between Regions and depends on network latency between Regions and cross-Region network traffic congestion. The RPO of Global Datastore is typically under one second, so the data written in a primary Region is available in secondary Regions within one second.
The RTO of Global Datastore for Redis is typically under a minute. Once a failover to a secondary cluster is initiated, ElastiCache typically promotes the secondary to full read and write capabilities in under a minute.

ElastiCache does not charge any premium to use Global Datastore for Redis. You pay for the primary and secondary caches in your Global Datastore.

3.3.8 Data tiering

Data tiering provides a new price-performance option for Redis workloads by using **lower-cost SSDs** in each cluster node in addition to storing data in memory.
It is ideal for workloads that regularly access up to 20% of their overall dataset and for applications that can tolerate additional latency when accessing data on SSD.
ElastiCache R6gd nodes with memory and SSDs have nearly 5x more total storage capacity and can help you achieve over 60% savings in price when running at maximum use compared to ElastiCache R6g nodes with memory only.

Data tiering works by automatically and transparently moving the least recently used items from memory to locally attached NVMe SSDs when available memory capacity is completely consumed.
When an item that moves to SSD is subsequently accessed, ElastiCache moves it back to memory asynchronously before serving the request.

Data tiering is designed to have minimal impact on application performance. Assuming 500-byte string values, you can expect an additional 300µs latency on average for requests to data stored on SSD compared to requests to data in memory.

ElastiCache for Redis supports data tiering for Redis versions 6.2 and above.

ElastiCache for Redis supports data tiering on Redis clusters using **R6gd nodes**.

All Redis commands and most ElastiCache features are supported when using data tiering.

There are no additional costs for using data tiering besides the node's hourly cost. Nodes with data tiering are available with on-demand pricing and as reserved nodes.

3.4 Memcached

You can cache a variety of objects using ElastiCache for Memcached. These objects include the content in persistent data stores to dynamically generated web pages to transient session data that might not require a persistent backing store. You can also use it to implement high-frequency counters to deploy admission control in high-volume web applications.

ElastiCache is an ideal front end for data stores like Amazon RDS or DynamoDB, providing a high-performance middle tier for applications with extremely high request rates or low latency requirements.

ElastiCache is protocol compliant with Memcached. Therefore, you can use standard Memcached operations in exactly the same way as you would in your existing Memcached deployments. ElastiCache supports both the text and binary protocols. It also supports most of the standard stats results, which can also be viewed as graphs with CloudWatch.
You can switch to using ElastiCache without recompiling or relinking your applications.

To configure the cache servers your application accesses, update your application's Memcached config file to include the endpoints of the servers (nodes) we provision for you.

You can access ElastiCache clusters in an Amazon VPC from either the Amazon EC2 network or from your own data center.
ElastiCache uses DNS entries to allow client applications to locate servers (nodes). The DNS name for a node remains constant, but the IP address of a node can change over time.

3.4.1 Configuration and scaling

with ElastiCache, you don't need to worry about getting the number of nodes exactly right, as you can quickly add or remove nodes later. You can also use ElastiCache Serverless to simplify running a highly available Memcached cache.

The amount of memory required is dependent upon the size of your data set and the access patterns of your application. To improve fault tolerance, once you have a rough idea of the total memory required, divide that memory into enough nodes such that your application can survive the loss of one or two nodes.

When creating a cluster or adding nodes to an existing cluster, you can choose the AZs for the new nodes. You can either specify the requested number of nodes in each AZ or select **Spread Nodes Across Zones**.

You can run a maximum of 300 nodes per Region.

The service will detect the node failure and react with the following automatic steps:

- ElastiCache will repair the node by acquiring new service resources and will then redirect the node's existing DNS name to point to the new service resources.
  For Amazon VPC installations, ElastiCache will ensure that both the DNS name and the IP address of the node remain the same when nodes

are recovered in case of failure. For non–Amazon VPC installations, ElastiCache will ensure that the DNS name of a node is unchanged; however, the underlying IP address of the node can change.

- If you associated an SNS topic with your cluster, when the new node is configured and ready to be used, ElastiCache will send an SNS notification to let you know that node recovery occurred.
  This allows you to optionally arrange for your applications to force the Memcached client library to attempt to reconnect to the repaired nodes.
  some Memcached libraries will stop using a server (node) indefinitely if they encounter communication errors or timeouts with that server

You could add more nodes to your existing Memcached Cluster by using the Add Node option on the Nodes tab for your Cache Cluster in the console or calling the ModifyCacheCluster API.

3.4.2 Compatibility

ElastiCache is ideally suited as a front end for AWS services like Amazon RDS and DynamoDB, providing extremely low latency for high-performance applications and offloading some of the request volume while these services provide long-lasting data durability.

Memcached client libraries are available for many, if not all of the popular programming languages.

ElastiCache does not require specific client libraries and works with existing Memcached client libraries without recompilation or application relinking.

3.4.3 Auto Discovery

Auto Discovery is a feature that saves developers time and effort, while reducing the complexity of their applications. Auto Discovery enables automatic discovery of cache nodes by clients when they are added to or removed from an ElastiCache cluster.

developers had to manually update the list of cache node endpoints. Depending on how the client application is architected, typically a client initialization is needed, resulting in downtime.
Through Auto Discovery, ElastiCache removes this complexity. With Auto Discovery, in addition to being backwards protocol compliant with the Memcached protocol, ElastiCache provides clients with information on cache cluster membership.A client capable of processing the additional information reconfigures itself, without any initialization, to use the most current nodes of an ElastiCache cluster.

An ElastiCache cluster can be created with nodes that are addressable through named endpoints. With Auto Discovery the ElastiCache cluster is also given a unique configuration endpoint, which is a DNS Record that is valid for the lifetime of the cluster. This DNS Record contains the DNS Names of the nodes that belong to the cluster.

You can connect to a read replica just as you would connect to a primary cache node. If you have multiple read replicas, it is up to your application to determine how read traffic will be distributed amongst them.

ElastiCache allows you to create **up to five** read replicas for a given primary cache node.

In the event of a failover, any associated and available read replicas should automatically resume replication once failover has completed.

Updates to a primary cache node will automatically be replicated to any associated read replicas. However, with Redis' asynchronous replication technology, a read replica can fall behind its primary cache node for a variety of reasons.
Typical reasons include:

- Write I/O volume to the primary cache node exceeds the rate at which changes can be applied to the read replica.
- Network partitions or latency between the primary cache node and a read replica.

Read replicas are subject to the strengths and weaknesses of Redis repli-

cation. If you are using read replicas, you should be aware of the potential for lag between a read replica and its primary cache node or "inconsistency." ElastiCache **emits a metric** to help you understand the inconsistency.

A read replica is billed as a standard cache node and at the same rates. Just like a standard cache node, the rate per cache node hour for a read replica is determined by the cache node class of the read replica.
ou are not charged for the data transfer incurred in replicating data between your primary cache node and read replica. Billing for a read replica begins as soon as the read replica has been successfully created.

Initiated failover is supported by ElastiCache so that you can resume cache operations as quickly as possible. When failing over, ElastiCache flips the DNS record for your cache node to point at the read replica, which is in turn promoted to become the new primary.
We encourage you to follow best practices and implement cache node connection retry at the application layer.

Your read replica can only be provisioned **in the same or different AZ** of the same Region as your cache node primary. You can, however, use the **Global Datastore for Redis** to work with fully managed, fast, reliable, and security-focused replication across AWS Regions.
Using this feature, you can create cross-Region read replica clusters for ElastiCache for Redis to enable **low-latency reads** and **disaster recovery** across AWS Regions.

You can gain visibility into the location of the current primary by using the console or DescribeCacheClusters API.

You can add or remove a read replica across one or more shards in a Redis Cluster environment. The cluster continues to stay online and serve incoming I/O during this operation.

3.4.4 Multi-AZ

Multi-AZ is a feature that allows you to run in a more highly available configuration when designing your own ElastiCache for Redis cache. All Elasti-

Cache Serverless caches are automatically run in a Multi-AZ configuration. An ElastiCache for Redis replication group consists of a primary and up to five read replicas. If Multi-AZ is enabled, then at least one replica is required per primary.
During certain types of planned maintenance, or in the unlikely event of an ElastiCache node failure or AZ failure, ElastiCache will automatically detect the failure of a primary, select a read replica, and promote it to become the new primary. ElastiCache also propagates the DNS changes of the promoted read replica, so if your application is writing to the primary node endpoint, no endpoint change will be needed.

The main benefits of running your ElastiCache for Redis in Multi-AZ mode are enhanced availability and a smaller need for administration.
When running ElastiCache in a Multi-AZ configuration, your caches are eligible for the 99.99% availability SLA. If an ElastiCache for Redis primary node failure occurs, the impact on your ability to read and write to the primary is limited to the time it takes for automatic failover to complete.
When Multi-AZ is enabled, ElastiCache node failover is automatic and requires no administration. You no longer need to monitor your Redis nodes and manually initiate a recovery in the event of a primary node disruption.

You can use Multi-AZ if you are using ElastiCache for Redis and have a replication group consisting of a primary node and one or more read replicas. If the primary node fails, ElastiCache will automatically detect the failure, select one from the available read replicas, and promote it to become the new primary.
ElastiCache will also spin up a new node to replace the promoted read replica in the same AZ of the failed primary. In case the primary failed due to temporary AZ disruption, the new replica will be launched once that AZ has recovered.

Placing both the primary and the replicas in the same AZ will not make your ElastiCache for Redis replication group resilient to an AZ disruption.

ElastiCache will fail over to a read replica in the event of any of the following:

- Loss of availability in primary's AZ

- Loss of network connectivity to primary
- Compute unit failure on primary

If there are more than one read replicas, the read replica with the smaller asynchronous replication lag to the primary will be promoted.

ElastiCache will create an event to inform you that automatic failover occurred.

AZs are engineered to provide low latency network connectivity to other AZs in the same Region. You should consider architecting your application and other AWS resources with redundancy across multiple AZs so your application will be resilient in the event of an AZ disruption.

3.4.5 Backup and restore

Backup and Restore is a feature that allows you to create snapshots of your ElastiCache for Redis caches. ElastiCache stores the snapshots, allowing users to subsequently use them to restore Redis caches.

Creating snapshots can be useful in case of data loss caused by node failure, as well as the unlikely event of a hardware failure.
When a backup is initiated, ElastiCache will take a snapshot of a specified Redis cache that can later be used for recovery or archiving. You can initiate a backup any time you choose or set a recurring daily backup with retention period of up to **35 days**.
ElastiCache for Redis snapshots are compatible with the open source Redis RDB file format.

Backup and Restore creates snapshots on a per-cache basis. Users can specify which ElastiCache for Redis cache to back up through the console, AWS CLI, or the ElastiCache API.
We recommend users enable backup on one of the cache's read replicas, minimizing any potential impact on the primary. When usin backups are automatically conducted against read replicas.

You can export your ElastiCache for Redis snapshots to an authorized S3 bucket in the same Region as your cache.

You must first copy your snapshot into an authorized S3 bucket of your choice in the same Region and then grant cross-account bucket permissions to the other account.

ElastiCache provides storage space for one snapshot free of charge for each active ElastiCache for Redis cache. Additional storage will be charged based on the space used by the snapshots with $0.085/GB every month. Data transfer for using the snapshots is free of charge.

When you delete an ElastiCache for Redis cache, your manual snapshots are retained. You will also have an option to create a final snapshot before the cache is deleted. Automatic cache snapshots are not retained.

3.4.6 Enhanced engine

The engine within ElastiCache for Redis is fully compatible with open source Redis but also comes with enhancements that improve performance, robustness, and stability. Some of the enhancements include:

- More usable memory: You can now safely allocate more memory for your application without risking increased swap usage during syncs and snapshots.
- Improved synchronization: More robust synchronization under heavy load and when recovering from network disconnections. Syncs are faster as both the primary and replicas no longer use the disk for this operation.
- Smoother failovers: In the event of a failover, your shard now recovers faster as replicas no longer flush their data to do a full resync with the primary.
- TLS offload and IO multiplexing: ElastiCache is designed to better use available CPU resources by handling certain network-related processes on dedicated threads.

The enhanced engine is fully compatible with open source Redis, thus you can enjoy its improved robustness and stability without the need to make any changes to your application code.

There is no additional charge for using the enhanced engine.

3.4.7 Encryption

Encryption at rest provides mechanisms to guard against unauthorized access of your data. When enabled, it encrypts the following aspects:

- Disk during sync, backup, and swap operations
- Backups stored in Amazon S3

ElastiCache for Redis offers default **encryption at rest**, as well as the ability to use your own symmetric customer-managed AWS KMS keys in AWS KMS.

The encryption in transit feature facilitates encryption of communications between clients and ElastiCache for Redis, as well as between the Redis servers.

Encryption in transit, encryption at rest, Redis AUTH, and Role-Based Access Control (RBAC) are features you can select when creating your ElastiCache for Redis cache. If you enabled in-transit encryption, you can choose to use Redis AUTH or RBAC for additional security and access control.

ElastiCache manages certification expiration and renewal behind the scenes. No user action is necessary for ongoing certificate maintenance.
There are no additional costs for using encryption.

3.4.8 Global Datastore

Global Datastore is a feature of ElastiCache for Redis that provides **fully managed**, **fast**, **reliable** and **security-focused** cross-Region replication. You can write to your ElastiCache for Redis cache in one Region and have the data available for read in **up to two other** cross-Region replica clusters, enabling **low-latency reads** and **disaster recovery** across Regions.

Global Datastore typically replicates data across Regions in **under one second**, increasing the responsiveness of your applications by providing geolocal reads closer to end users.
In the unlikely event of Regional degradation, one of the healthy cross-Region replica caches can be promoted to become the primary with full read and write capabilities.
The promotion typically completes in less than a minute.

You can replicate to **up to two** secondary Regions within a Global Datastore. The caches in secondary Regions can be used to serve low-latency local reads and for disaster recovery.

Global Datastore is supported on **ElastiCache for Redis 5.0.6** onward.
You can set up a Global Datastore by using an existing cache or creating a new cache to be used as a primary.
There is support for Global Datastore in AWS CloudFormation.

ElastiCache doesn't automatically promote a secondary cluster in the event when a primary cluster is degraded.
You can **manually initiate** the failover by promoting a secondary cluster to become a primary.

Global Datastore uses encryption in transit for cross-Region traffic to keep your data more secure.
You can also encrypt your primary and secondary caches using encryption at rest to keep your data better secured.
Each primary and secondary cache can have a separate customer-managed AWS KMS key for encryption at rest.

ElastiCache doesn't provide an SLA for Recovery Point Objective and Recovery Time Objective.
The RPO varies based on **replication lag** between Regions and depends on **network latency** between Regions and cross-Region network traffic congestion.
The RPO of Global Datastore is typically under one second.
The RTO of Global Datastore for Redis is typically under a minute.

ElastiCache does not charge any premium to use Global Datastore for

Redis.

3.4.9 Data tiering

Data tiering provides a new price-performance option for Redis workloads by using lower-cost SSDs in each cluster node in addition to storing data in memory.
It is ideal for workloads that regularly access **up to 20% of their overall dataset** and for applications that can **tolerate additional latency** when accessing data on SSD.

Data tiering works by automatically and transparently moving the least recently used items from memory to locally attached NVMe SSDs when available memory capacity is **completely consumed**.

Data tiering is designed to have minimal impact on application performance.

ElastiCache for Redis supports data tiering for **Redis versions 6.2** and above. ElastiCache for Redis supports data tiering on Redis clusters using **R6gd nodes**. All Redis commands and most ElastiCache features are supported when using data tiering.

There are no additional costs for using data tiering besides the node's hourly cost.

3.5 Memcached

You can cache a variety of objects using ElastiCache for Memcached. These objects include the content in persistent data stores to dynamically generated web pages, to transient session data that might not require a persistent backing store.
You can also use it to implement high-frequency counters to deploy admission control in high-volume web applications.

ElastiCache is an ideal front end for **data stores**, providing a high-performance middle tier for applications with extremely high request rates or low latency requirements.

ElastiCache is protocol compliant with Memcached. You can use standard Memcached operations like get, set, incr, and decr in exactly the same way as you would in your existing Memcached deployments. ElastiCache supports both the text and binary protocols. It also supports most of the standard stats results, which can also be viewed as graphs.
You can switch to using ElastiCache without recompiling or relinking your applications.
To configure the cache servers your application accesses, update your application's Memcached config file to include the endpoints of the servers (nodes) we provision for you.

You can access ElastiCache clusters in an Amazon VPC from either the Amazon EC2 network or from your own data center.
ElastiCache uses DNS entries to allow client applications to locate servers (nodes). The DNS name for a node remains constant, but the **IP address of a node can change** over time,

3.5.1 Configuration and scaling

With ElastiCache, you don't need to worry about getting the number of nodes exactly right, as you can quickly add or remove nodes later.
You can also use ElastiCache Serverless to simplify running a highly available Memcached cache.
The following two inter-related aspects can be considered for the choice of your initial configuration:

- The total memory required for your data to achieve your target cache-hit rate
- The number of nodes required to maintain acceptable application performance without overloading the database backend in the event of node failures.

The amount of memory required is dependent upon the size of your data set and the access patterns of your application.

Once you have a rough idea of the total memory required, divide that memory into enough nodes such that your application can survive the loss of one or two nodes. It is important that other systems such as databases will not be overloaded if the cache-hit rate is temporarily reduced during failure recovery of one or more nodes.

When creating a cluster or adding nodes to an existing cluster, you can choose the AZs for the new nodes.
You can either specify the requested number of nodes in each AZ or select Spread Nodes Across Zones.
Nodes can only be placed in AZs that are part of the selected cache subnet group.

You can run a maximum of 300 nodes per Region.

The service will detect the node failure and react with the following automatic steps:

- ElastiCache will repair the node by acquiring new service resources and will then redirect the node's existing DNS name to point to the new service resources.
- If you associated an SNS topic with your cluster, when the new node is configured and ready to be used, ElastiCache will send an SNS notification to let you know that node recovery occurred.

You could add more nodes to your existing Memcached Cluster by using the Add Node option.

3.5.2 Compatibility

ElastiCache is ideally suited as a front end for AWS services, providing extremely low latency for high-performance applications and offloading some of the request volume while these services provide long-lasting data durability. Memcached client libraries are available for many, if not all of the popular programming languages.

ElastiCache **does not require** specific client libraries and works with existing Memcached client libraries without recompilation or application relinking (Memcached 1.4.5 and later).

3.5.3 Auto Discovery

Auto Discovery enables automatic discovery of cache nodes by clients when they are added to or removed from an ElastiCache cluster.
Previously, to handle cluster membership changes, developers had to manually update the list of cache node endpoints. Depending on how the client application is architected, typically a client initialization is needed, resulting in downtime.
Through Auto Discovery, ElastiCache removes this complexity. With Auto Discovery, in addition to being backwards protocol compliant with the Memcached protocol, ElastiCache provides clients with information on cache cluster membership. A client capable of processing the additional information reconfigures itself, without any initialization, to use the most current nodes of an ElastiCache cluster.

An ElastiCache cluster can be created with nodes that are addressable through **named endpoints**. With Auto Discovery the ElastiCache cluster is also given a **unique configuration endpoint**, which is a DNS Record that is valid for the lifetime of the cluster.
This DNS Record contains the DNS Names of the nodes that belong to the cluster. ElastiCache will ensure that the configuration endpoint always points to **at least one such target node**.
A query to the target node then returns endpoints for all the nodes of the cluster in question. After this, you can connect to the cluster nodes just as before and use the Memcached protocol commands.
To use Auto Discovery, you will need a client **capable of Auto Discovery**. Upon initialization, the client will automatically determine the current members of the ElastiCache cluster using the configuration endpoint. When you make changes to your cache cluster by adding or removing nodes, or if a node is replaced upon failure, the Auto Discovery client automatically determines the changes.

You will not get the Auto Discovery feature with the existing Memcached

clients. To use the Auto Discovery feature a client must be able to use a configuration endpoint and determine the cluster node endpoints.

To take advantage of Auto Discovery, a client capable of Auto Discovery must be used to connect to an ElastiCache Cluster.

ElastiCache is still Memcached protocol compliant and does not require you to change your clients. However, to take advantage of the Auto Discovery feature, we **enhanced the Memcached client capabilities**. If you choose not to use the ElastiCache Cluster Client, you can continue to use your own clients or modify your own client library.

the same ElastiCache cluster can be connected through a client capable of Auto Discovery and the traditional Memcached client **at the same time**. ElastiCache remains 100% Memcached compliant.

You can stop using Auto Discovery at any time. You can disable Auto Discovery by specifying the mode of operation during the ElastiCache Cluster client initialization. Also, since ElastiCache continues to support Memcached, you can use any Memcached protocol-compliant client as before.

3.5.4 Engine version management

ElastiCache allows you to control if and when the Memcached protocol-compliant software powering your cluster is upgraded to new versions supported by ElastiCache.
This provides you with the flexibility to **maintain compatibility** with specific Memcached versions, test new versions with your application before deploying in production, and **perform version upgrades** on your own terms and timelines.
Version upgrades involve some compatibility risk; thus they will not occur automatically and must be initiated by you. This approach to software patching puts you in the driver's seat of version upgrades, but still offloads the work of patch application to ElastiCache.
We can patch your cluster on your behalf if we determine there is any security vulnerability in the system or cache software.

You can specify any currently supported version (minor or major) when

creating a new cluster. If you wish to initiate an upgrade to a supported engine version release, you can do so using the Modify option for your cluster. Specify the version you wish to upgrade to in the Cache Engine Version field. The upgrade will then be applied on your behalf either immediately or during the next scheduled maintenance window for your cluster.

You can test your cluster against a new version before upgrading by creating a new cluster with the new engine version. You can point your development or staging application to this cluster, test it, and decide whether or not to upgrade your original cluster.

We plan to support additional Memcached versions for ElastiCache, both major and minor. The number of new version releases supported in a given year will vary based on the frequency and content of the Memcached version releases and the outcome of a thorough vetting of the release by our engineering team.

You can upgrade your existing Memcached cluster by using the Modify process. When upgrading from an older version of Memcached to Memcached version 1.4.33 or newer, please ensure that your existing parameter **max_chunk_size** values satisfies conditions needed for **slab_chunk_max** parameter.

4 Amazon Kinesis Data Streams

With Kinesis Data Streams, you can build custom applications that process or analyze **streaming data** for specialized needs. You can add various types of data such as clickstreams, application logs, and social media to a Kinesis data stream from hundreds of thousands of sources.
Within seconds, the data will be available for your applications to read and process from the stream.

Kinesis Data Streams manages the infrastructure, storage, networking, and configuration needed to stream your data at the level of your data throughput. You don't have to worry about provisioning, deployment, or ongoing maintenance of hardware, software, or other services for your data streams.

Kinesis Data Streams synchronously replicates data across three Availability Zones, providing high availability and data durability.
Kinesis Data Streams **scales capacity automatically**, freeing you from provisioning and managing capacity.
You can choose **provisioned mode** if you want to provision and manage throughput on your own.

Kinesis Data Streams is useful for rapidly **moving data off data producers** and then **continuously processing the data**
The following are typical scenarios for using Kinesis Data Streams:

- **Accelerated log and data feed intake**: Instead of waiting to batch the data, you can have your data producers push data to a Kinesis data stream as soon as the data is produced, preventing data loss in case of producer failure.
- **Real-time metrics and reporting**: You can extract metrics and generate reports from Kinesis data stream data in real time.
- **Real-time data analytics**: You can run real-time streaming data analytics.
- **Log and event data collection**: Collect log and event data from sources such as servers, desktops, and mobile devices. You can then build applications using AWS services to continuously process the data.
- **Power event-driven applications**: Quickly pair with AWS Lambda to respond or adjust to immediate occurrences within the event-driven applications in your environment, at any scale.

4.1 Key concepts

A **shard** has a sequence of data records in a stream. It serves as a **base throughput unit** of a Kinesis data stream.
A shard supports **1 MB/second and 1,000 records per second** for writes and **2 MB/second** for reads. The shard limits ensure predictable performance, making it easy to design and operate a highly reliable data streaming workflow. A producer puts data records into shards and a consumer gets data records from shards. Consumers use shards for **parallel data processing**

and for consuming data in the exact order in which they are stored.
If writes and reads exceed the shard limits, the producer and consumer applications will receive throttles, which can be handled through retries.

A record is the unit of data stored in an Amazon Kinesis data stream. A record is composed of a sequence number, partition key, and data blob.
Data blob is the data of interest your data producer adds to a data stream.
The **maximum size of a data blob is 1 MB**.

A partition key is used to **isolate and route records** to different shards of a data stream. A partition key is specified by your data producer while adding data to a Kinesis data stream.

A sequence number is a **unique identifier for each record**. Sequence number is assigned by Amazon Kinesis when a data producer calls PutRecord or PutRecords operation to add data to a Amazon Kinesis data stream. Sequence numbers for the same partition key generally increase over time.
The longer the time period between PutRecord or PutRecords requests, the larger the sequence numbers become.

The capacity mode of Kinesis Data Streams determines how capacity is managed and usage is charged for a data stream. You can choose between provisioned and on-demand modes.
In provisioned mode, **you specify the number of shards** for the data stream. The total capacity of a data stream is the sum of the capacities of its shards. You can increase or decrease the number of shards in a data stream as needed, and you **pay for the number of shards** at an hourly rate.
In on-demand mode, **AWS manages the shards** to provide the necessary throughput. You pay only for the **actual throughput** used, and Kinesis Data Streams automatically accommodates your workload throughput needs.
All Kinesis Data Streams write and read APIs, along with optional features such as Extended Retention and Enhanced Fan-Out, are supported in both capacity modes.

On-demand mode is best suited for workloads with unpredictable and highly variable traffic patterns. You should use this mode if you prefer AWS to manage capacity on your behalf or prefer pay-per-throughput pricing.
Provisioned mode is best suited for predictable traffic, where capacity re-

quirements are easy to forecast. You should consider using provisioned mode if you want fine-grained control over how data is distributed across shards. Provisioned mode is also suitable if you want to provision additional shards so the consuming application can have more read throughput to speed up the overall processing.

You can **switch between** on-demand and provisioned mode **twice a day**. The shard count of your data stream **remains the same** when you switch from provisioned mode to on-demand mode and vice versa.
With the switch from provisioned to on-demand capacity mode, your data stream retains **whatever shard count it had before** the transition.

4.2 Adding data to Kinesis Data Streams

You can add data to a Kinesis data stream through PutRecord and PutRecords operations, KPL, or Amazon Kinesis Agent.
PutRecord operation allows a single data record within an API call, and PutRecords operation allows multiple data records within an API call.
KPL is an easy-to-use and highly configurable library that helps you put data into an Amazon Kinesis data stream.

Amazon Kinesis Agent is a prebuilt Java application that offers an easy way to collect and send data to your Amazon Kinesis data stream. You can install the agent on Linux-based server environments.
The agent monitors certain files and continuously sends data to your data stream.

Your data blob, partition key, and data stream name are required parameters of a PutRecord or PutRecords call. The size of your data bloab and partition key will be counted against the data throughput of your Amazon Kinesis data stream, which is determined by the number of shards within the data stream.

4.3 Reading and processing data from Kinesis Data Streams

A consumer is an application that **processes all data from a Kinesis data stream**. You can choose between **shared fan-out** and **enhanced fan-out** consumer types to read data from a Kinesis data stream.
The shared fan-out consumers all share a **shard's 2 MB/second of read throughput** and **five transactions per second limits** and require the use of the GetRecords API.
An enhanced fan-out consumer gets its **own 2 MB/second allotment of read throughput**, allowing multiple consumers to read data from the same stream in parallel.
We recommend using enhanced fan-out consumers if you want to add more than one consumer to your data stream.

You can use managed services such as AWS Lambda, Amazon Managed Service for Apache Flink, and AWS Glue to process data stored in Kinesis Data Streams. These managed services take care of provisioning and managing the underlying infrastructure so you can focus on writing your business logic.
You can also deliver data stored in Kinesis Data Streams to Amazon S3, Amazon OpenSearch Service, Amazon Redshift, and custom HTTP endpoints using its prebuilt integration with Kinesis Data Firehose. You can also build custom applications using Amazon Kinesis Client Library.

KCL handles complex issues such as adapting to changes in data stream volume, load-balancing streaming data, coordinating distributed services, and processing data with fault tolerance. KCL enables you to focus on business logic while building applications.

The SubscribeToShard API is a high-performance streaming API that pushes data from shards to consumers over a persistent connection without a request cycle from the client. The SubscribeToShard API uses the HTTP/2 protocol to deliver data to registered consumers whenever new data arrives on the shard, typically within 70 milliseconds, offering approximately 65% faster delivery compared to the GetRecords API.

Enhanced fan-out is an optional feature for Kinesis Data Streams con-

sumers that provides logical 2 MB/second throughput pipes between consumers and shards. This allows you to scale the number of consumers reading from a data stream in parallel, while maintaining high performance.

You should use enhanced fan-out if you have, or expect to have, multiple consumers retrieving data from a stream in parallel, or if you have at least one consumer that requires the use of the SubscribeToShard API to provide sub-200 millisecond data delivery speeds between producers and consumers.

Consumers use enhanced fan-out by retrieving data with the SubscribeToShard API. The name of the registered consumer is used within the SubscribeToShard API, which leads to utilization of the enhanced fan-out benefit provided to the registered consumer.

You can have multiple consumers using enhanced fan-out and others not using enhanced fan-out at the same time.

To use SubscribeToShard, you need to register your consumers, which activates enhanced fan-out. By default, your consumer will use enhanced fan-out automatically when data is retrieved through SubscribeToShard.

4.4 On-demand mode

A new data stream created in on-demand mode has a quota of **4 MB/second** and **4,000 records per second** for writes. By default, these streams automatically scale up to **200 MB/second** and **200,000 records per second** for writes.

A data stream in on-demand mode accommodates up to double its previous peak write throughput observed in the last 30 days. As your data stream's write throughput hits a new peak, Kinesis Data Streams **scales the stream's capacity automatically**.
You will see **ProvisionedThroughputExceeded** exceptions if your traffic grows more than double the previous peak within a 15-minute duration.

On-demand mode's aggregate read capacity increases proportionally to write throughput to ensure that consuming applications always have ade-

quate read throughput to process incoming data in real time.
You get at least twice the write throughput to read data using the GetRecords API. We recommend using one consumer with the GetRecord API so it has enough room to catch up when the application needs to recover from downtime. To add more than one consuming application, you need to use enhanced fan-out, which supports adding up to 20 consumers to a data stream using the SubscribeToShard API, with each having dedicated throughput.

4.5 Provisioned mode

The throughput of a Kinesis data stream in provisioned mode is designed to scale without limits by increasing the number of shards within a data stream.

You can scale up a Kinesis data stream capacity in provisioned mode by splitting existing shards using the SplitShard API. You can scale down capacity by merging two shards using the MergeShard API. Alternatively, you can use UpdateShardCount API to scale a stream capacity to a specific shard count.

The throughput of a Kinesis data stream is determined by the number of shards within the data stream.
Follow the steps below to estimate the initial number of shards your data stream needs in provisioned mode: Estimate the average size of the record written to the data stream in kilobytes (KB), rounded up to the nearest 1 KB. (average_data_size_in_KB)

Estimate the number of records written to the data stream per second. (number_of_records_per_second)

Decide the number of Amazon Kinesis Applications consuming data concurrently and independently from the data stream. (number_of_consumers)

Calculate the incoming write bandwidth in KB (incoming_write_bandwidth_in_KB), which is equal to the **average_data_size_in_KB multiplied by the number_of_records_per_second**.

Calculate the outgoing read bandwidth in KB (outgoing_read_bandwidth_in_KB),

which is equal to the **incoming_write_bandwidth_in_KB multiplied by the number_of_consumers**.

You can then calculate the initial number of shards (number_of_shards) your data stream needs using the following formula:
number_of_shards = max (incoming_write_bandwidth_in_KB/1000, outgoing_read_bandwidth_in_KB/2000)

The throughput of a Kinesis data stream is designed to scale without limits. The default shard quota is **500 shards** per stream for the following AWS Regions: **US East (N. Virginia)**, **US West (Oregon)**, and **Europe (Ireland)**. For all other Regions, the default shard quota is 200 shards per stream.

In provisioned mode, the capacity limits of a Kinesis data stream are defined by the number of shards within the data stream. The limits can be exceeded either by data throughput or by the number of PUT records.
While the capacity limits are exceeded, the put data call will be rejected with a **ProvisionedThroughputExceeded** exception. If this is due to a temporary rise of the data stream's input data rate, retry by the data producer will eventually lead to completion of the requests.
If it's due to a sustained rise of the data stream's input data rate, you should **increase the number of shards** within your data stream to provide enough capacity for the put data calls to consistently succeed.

In provisioned mode, the capacity limits of a Kinesis data stream are defined by the number of shards within the data stream. The limits can be exceeded either by data throughput or by the number of read data calls. While the capacity limits are exceeded, the read data call will be rejected with a **ProvisionedThroughputExceeded** exception.
If this is due to a temporary rise of the data stream's output data rate, retry. If it's due to a sustained rise of the data stream's output data rate, you should increase the number of shards within your data stream to provide enough capacity.

4.6 Extended and long-term data retention

The default retention period of 24 hours covers scenarios where intermittent lags in processing require catch-up with the real-time data. A seven-day retention lets you reprocess data for up to seven days to resolve potential downstream data losses. Long-term data retention greater than seven days and up to 365 days lets you reprocess old data.

You can use the same **getShardIterator, GetRecords, and SubscribeToShard** APIs to read data retained for more than seven days. The consumers can move the iterator to the desired location in the stream, retrieve the shard map and read the records.

There are API enhancements to **ListShards, GetRecords, and SubscribeToShard** APIs. You can use the new filtering option with the **TimeStamp parameter** available in the ListShards API to efficiently retrieve the shard map and improve the performance of reading old data.
The filter lets applications discover and enumerate shards from the point in time you wish to reprocess data and eliminate the need to start at the trim horizon.
GetRecords and SubscribeToShards have a new field, **ChildShards**, which allows you to quickly discover the children shards when an application finishes reading data from a closed shard, instead of having to traverse the shard map again. The fast discovery of shards makes efficient use of the consuming application's compute resources for any sized stream, irrespective of the data retention period.

You should consider the API enhancements if you plan to retain data longer and scale your stream's capacity regularly. Stream scaling operations **close existing shards** and open new child shards. The data in all the open and closed shards is **retained until the end of the retention period**.
The total number of shards increase linearly with a longer retention period and multiple scaling operations.

Clients of Kinesis Data Streams can use the AWS Glue Schema Registry, a serverless feature of AWS Glue.
The Schema Registry is available at no additional charge.

4.7 Managing Kinesis Data Streams

There are two ways to change the throughput of your data stream. You can use the UpdateShardCount API or the AWS Management Console or you can change the throughput of an Amazon Kinesis data stream by adjusting the number of shards within the data stream

Typical scaling requests should take a few minutes to complete. Larger scaling requests will take longer than smaller ones.

You can continue adding data to and reading data from your Kinesis data stream while you use UpdateShardCount or reshard to change the throughput of the data stream or when Kinesis Data Streams does it automatically in on-demand mode.

Kinesis Data Streams integrates with AWS Identity and Access Management (IAM).

You can use IAM assume role or resource-based policy to share access with another account.

Kinesis Data Streams integrates with Amazon CloudTrail, a service that records AWS API calls for your account and delivers log files to you.

Kinesis Data Streams allows you to tag your Kinesis data streams for easier resource and cost management. A tag is a user-defined label expressed as a key-value pair that helps organize AWS resources.

4.8 Security

Amazon Kinesis is secure by default. Only the account and data stream owners have access to the Kinesis resources they create.
Kinesis supports user authentication to control access to data. You can use IAM policies to selectively grant permissions to users and groups of users.
You can securely put and get your data from Kinesis through SSL endpoints using the HTTPS protocol. If you need extra security, you can use server-side encryption with AWS KMS keys to encrypt data stored in your data stream.

Lastly, you can use your own encryption libraries to encrypt data on the client side before putting the data into Kinesis.

You can privately access Kinesis Data Streams APIs from your Amazon VPC by creating VPC Endpoints. With VPC Endpoints, the routing between the VPC and Kinesis Data Streams is handled by the AWS network without the need for an internet gateway, NAT gateway, or VPN connection.

You can use server-side encryption, which is a fully managed feature that automatically encrypts and decrypts data as you put and get it from a data stream. You can also write encrypted data to a data stream by encrypting and decrypting on the client side.

You might choose server-side encryption over client-side encryption for any of the following reason:

- It is hard to enforce client-side encryption.
- They want a second layer of security on top of client-side encryption.
- It is hard to implement client-side key management schemes.

Server-side encryption for Kinesis Data Streams automatically encrypts data using a user specified AWS KMS key before it is written to the data stream storage layer, and decrypts the data after it is retrieved from storage. Encryption makes writes impossible and the payload and the partition key unreadable unless the user writing or reading from the data stream has the permission to use the key selected for encryption on the data stream.

With server-side encryption your client-side applications do not need to be aware of encryption, they do not need to manage KMS keys or cryptographic operations, and your data is encrypted when it is at rest and in motion within the Kinesis Data Streams service. All KMS keys used by the server-side encryption feature are provided by the AWS KMS.

If you use the AWS-managed KMS key for Kinesis (key alias = aws/kinesis) your applications will not be impacted by enabling or disabling encryption with this key.

If you use a different KMS key, like a custom AWS KMS key or one you imported into the AWS KMS service, and if your producers and consumers of a data stream do not have permission to use the KMS key used for encryption, then your PUT and GET requests will fail. Before you can use server-side encryption you must configure AWS KMS key policies to allow encryption and decryption of messages.

If you are using the AWS-managed KMS key for Kinesis and are not exceeding the AWS Free Tier KMS API usage costs, your use of server-side encryption is free.

Kinesis Data Streams server-side encryption is available in the AWS GovCloud Region and all public Regions except the China (Beijing) Region.

Kinesis Data Streams uses an **AES-GCM 256** algorithm for encryption.

Only new data written into the data stream will be encrypted (or left decrypted) by the new application of encryption.

Server-side encryption encrypts the payload of the message along with the partition key, which is specified by the data stream producer applications. Server-side encryption is a stream specific feature.

Kinesis Data Streams is **not** currently available in the AWS Free Tier.

4.9 Service Level Agreement

Our Kinesis Data Streams SLA guarantees a Monthly Uptime Percentage of at least 99.9% for Kinesis Data Streams.

You are eligible for a SLA credit for Kinesis Data Streams under the Kinesis Data Streams SLA if more than one Availability Zone in which you are running a task, within the same Region has a Monthly Uptime Percentage of less than 99.9% during any monthly billing cycle.

4.10 Pricing and billing

Kinesis Data Streams uses simple pay-as-you-go pricing. There are no up-front costs or minimum fees, and you pay only for the resources you use. Kinesis Data Streams has two capacity modes, **on demand** and **provisioned**, and both come with specific billing options.

With on-demand capacity mode, you don't need to specify how much read and write throughput you expect your application to perform. In this mode, pricing is based on the volume of data ingested and retrieved along with a per-hour charge for each data stream in your account.

With provisioned capacity mode, you specify the number of shards necessary for your application based on its write and read request rate. A shard is a unit of capacity that provides 1 MB/second of write and 2 MB/second of read throughout.
You're charged for each shard at an hourly rate. You also pay for records written into your Kinesis data stream.

A consumer-shard hour is calculated by multiplying the number of registered stream consumers with the number of shards in the stream. You will also pay only for the prorated portion of the hour the consumer was registered to use enhanced fan-out.

Kinesis data stream capture, process and store stream data
Kinesis firestore kiad data stream into AWS data store
Kinesis store immutable data
Kinesis data stream work on provisioned and ondemand mode for provisioning stored data shards
Kinesis work on a shared fan-out consumer or enanched consumer.
Shard splitting is used to manage "**hot shard**"
Standard shard have provision of 2MB per shard while enanched consumer have provision of 2MB per consumer per shard.

Decrease the frequency size of the request could help to avoid spike on kinesis streams

5 AWS Lambda

AWS Lambda lets you run code without provisioning or managing servers. You pay only for the compute time you consume - there is no charge when your code is not running.
You can run code for virtually any type of application or backend service - all with zero administration.
Lambda takes care of everything required to run and scale your code with high availability.

Serverless computing allows you to build and run applications and services without thinking about servers.
Your application still runs on servers, but all the server management is done by AWS.

AWS Lambda makes it easy to execute code in response to events. With Lambda, you do not have to provision your own instances; Lambda performs all the operational and administrative activities on your behalf.
AWS Lambda provides easy scaling and high availability to your code without additional effort on your part.

AWS Lambda offers an easy way to accomplish many activities in the cloud.
AWS Lambda natively supports Java, Go, PowerShell, Node.js, C#, Python, and Ruby code, and provides a Runtime API which allows you to use any additional programming languages to author your functions.

AWS Lambda operates the compute infrastructure **on your behalf**, allowing it to perform health checks, apply security patches, and do other routine maintenance.

Each AWS Lambda function runs in its own **isolated environment**, with its own resources and file system view.

AWS Lambda stores code in Amazon S3 and encrypts it at rest. AWS Lambda performs additional integrity checks while your code is in use.

5.1 AWS Lambda Functions

The code you run on AWS Lambda is uploaded as a "Lambda function". Each function has associated configuration information.
The code must be written in a "stateless" style.
Local file system access, child processes, and similar artifacts may not extend beyond the lifetime of the request, and any persistent state should be stored in Amazon S3, Amazon DynamoDB, Amazon EFS, or another Internet-available storage service.
Lambda functions **can include libraries**, even native ones.

AWS Lambda may choose to retain an instance of your function and reuse it to serve a subsequent request, rather than creating a new copy.

You can configure each Lambda function with its own ephemeral storage between **512MB and 10,240MB**, in 1MB increments.
The ephemeral storage is available in each function's **/tmp** directory.
Each function has access to 512MB of storage at no additional cost. When configuring your functions with more than 512MB of ephemeral storage, you will be charged based on the amount of storage you configure, and how long your function runs, metered in 1ms increments.

All data stored in ephemeral storage is **encrypted at rest** with a key managed by AWS.

You can use AWS CloudWatch Lambda Insight metrics to monitor your ephemeral storage usage.

If your application needs durable, persistent storage, consider using Amazon S3 or Amazon EFS. If your application requires storing data needed by code in a single function invocation, consider using AWS Lambda ephemeral storage as a transient cache.

If you application needs persistent storage, consider using Amazon EFS or Amazon S3. When you enable Provisioned Concurrency for your function, your function's initialization code runs during allocation and every few hours, as running instances of your function are recycled. You can see the initialization time in logs and traces after an instance processes a request.

Initialization is billed even if the instance never processes a request. This Provisioned Concurrency initialization behavior may affect how your function interacts with data you store in ephemeral storage, even when your function isn't processing requests.

Keeping functions stateless enables AWS Lambda to rapidly launch as many copies of the function as needed to scale to the rate of incoming events. While AWS Lambda's programming model is stateless, your code can access stateful data by calling other web services.

AWS Lambda allows you to use normal language and operating system features, such as creating additional threads and processes. Resources allocated to the Lambda function, including memory, execution time, disk, and network use, must be shared among all the threads/processes it uses.

Inbound network connections are blocked by AWS Lambda, and for outbound connections, **only TCP/IP and UDP/IP sockets are supported**, and ptrace (debugging) system calls are blocked.
TCP port 25 traffic is also **blocked as an anti-spam measure**.

You can package your code as a ZIP and upload it using the AWS Lambda console from your local environment or specify an Amazon S3 location where the ZIP file is located. Uploads must be **no larger than 50MB** (compressed).

You can easily create and modify environment variables from AWS.
For sensitive information, such as database passwords, we recommend you use **client-side encryption** using AWS Key Management Service and store the resulting values as ciphertext in your environment variable.
You will need to include logic in your AWS Lambda function code to decrypt these values.

Important

You can adjust and secure the resources associated with your Lambda function using the Lambda API or console.

You can package any code (frameworks, SDKs, libraries, and more) as a Lambda Layer and manage and share them easily across multiple functions.

AWS Lambda automatically monitors Lambda functions on your behalf, reporting real-time metrics through Amazon CloudWatch.
You can view statistics for each of your Lambda functions via the Amazon CloudWatch console or through the AWS Lambda console.

AWS Lambda automatically integrates with Amazon CloudWatch logs, creating a log group for each Lambda function and providing **basic application lifecycle event log entries**, including logging the resources consumed for each use of that function.
You can also call third-party logging APIs in your Lambda function.

AWS Lambda **scales them automatically on your behalf**. Every time an event notification is received for your function, AWS Lambda quickly locates free capacity within its compute fleet and runs your code. Since your code is stateless, AWS Lambda can **start as many copies of your function as needed** without lengthy deployment and configuration delays. There are no fundamental limits to scaling a function.
AWS Lambda will **dynamically allocate capacity** to match the rate of incoming events.

You choose the amount of memory you want for your function, and are allocated proportional CPU power and other resources.
You can set your memory from **128MB to 10,240MB**.

Customers running memory or compute-intensive workloads can now use more memory for their functions. Larger memory functions help multi-threaded applications run faster.

AWS Lambda functions can be configured to run up to 15 minutes per execution. You can set the timeout to **any value between 1 second and 15 minutes**.

AWS Lambda is priced on a pay-per-use basis.

In addition to saving money on Amazon EC2 and AWS Fargate, you can

also use Compute Savings Plans to save money on AWS Lambda. Compute Savings Plans offer **up to 17% discount** on Duration, Provisioned Concurrency. Compute Savings Plans do not offer a discount on Requests in your Lambda bill.
However, your Compute Savings Plans commitment can apply to Requests at regular rates.

AWS Lambda function has a single, current version of the code. Clients of your Lambda function can **call a specific version** or get the latest implementation.

Deployment times may vary with the size of your code, but AWS Lambda functions are typically ready to call within **seconds of upload**.

You can include your own copy of a library in order to use a different version than the default one provided by AWS Lambda.

AWS Lambda offers discounted pricing tiers for monthly on-demand function duration above certain thresholds. Tiered pricing is available for functions running on both x86 and Arm architectures. Lambda pricing tiers are applied to aggregate monthly on-demand duration of your functions running on the same architecture, in the same region, within the account.
If you're using consolidated billing in AWS Organizations, pricing tiers are applied to the aggregate monthly duration of your functions running on the same architecture, in the same region, across the accounts in the organization.

ambda usage that is covered by your hourly savings plan commitment is billed at the applicable CSP rate and discount. The remaining usage that is not covered by this commitment will be billed at the rate corresponding to the tier your monthly aggregate function duration falls in.

5.2 Using AWS Lambda to process AWS events

An event source is an AWS service or developer-created application that **produces events that trigger an AWS Lambda function** to run. Some services publish these events to Lambda by invoking the cloud function directly.

Lambda can also poll resources in other services that do not publish events to Lambda.
Many other services, can act as event sources simply by logging to Amazon S3 and using S3 bucket notifications to trigger AWS Lambda functions.

Events are passed to a Lambda function as an **event input parameter**. For event sources where events arrive in batches, the event parameter may contain multiple events in a single call, based on the batch size you request.

From the AWS Lambda console, you can select a function and associate it with notifications from an Amazon S3 bucket. Alternatively, you can use the Amazon S3 console and configure the bucket's notifications to send to your AWS Lambda function.

You can trigger a Lambda function on DynamoDB table updates by subscribing your Lambda function to the DynamoDB Stream associated with the table.

You can invoke a Lambda function using a custom event through AWS Lambda's invoke API. Only the function owner or another AWS account that the owner has granted permission can invoke the function.

AWS Lambda is designed to process events **within milliseconds**. Latency will be higher immediately after a Lambda function is created, updated, or if it has not been used recently.

You upload the code you want AWS Lambda to execute and then invoke it from your mobile app using the AWS Lambda SDK included in the AWS Mobile SDK. You can make both direct (synchronous) calls to retrieve or check data in real time, as well as asynchronous calls. You can also define a custom API using Amazon API Gateway and invoke your Lambda functions through any REST compatible client.

You can invoke a Lambda function over HTTPS by **defining a custom RESTful API** using Amazon API Gateway. This gives you an endpoint for your function which can respond to REST calls.

When your app uses the Amazon Cognito identity, end users can authen-

ticate themselves using a variety of public login providers.
User identity is then automatically and secured presented to your Lambda function in the form of an Amazon Cognito id, allowing it to access user data from Amazon Cognito, or as a key to store and retrieve data in Amazon DynamoDB or other web services.

For Amazon S3 bucket notifications and custom events, AWS Lambda will attempt execution of your function **three times** in the event of an error condition in your code or if you **exceed a service or resource limit**.
For ordered event sources that AWS Lambda polls on your behalf, Lambda will continue attempting execution in the event of a developer code error **until the data expires**.
You can also set Amazon CloudWatch alarms based on error or execution throttling rates.

5.3 Using AWS Lambda to build applications

Lambda-based applications are composed of functions **triggered by events**.
A typical serverless application consists of one or more functions triggered by events. These functions can stand alone or leverage other resources.
The most basic serverless application is simply a function.

You can deploy and manage your serverless applications using the AWS **Serverless Application Model**. AWS SAM is a specification that prescribes the rules for expressing serverless applications on AWS.
This specification aligns with the syntax used by AWS CloudFormation today and is supported natively within AWS CloudFormation as a set of serverless resources.
These resources make it easier for AWS customers to use CloudFormation to configure and deploy serverless applications using existing CloudFormation APIs.

You can use AWS Step Functions to coordinate a series of AWS Lambda functions in a specific order. You can invoke multiple Lambda functions sequentially, passing the output of one to the other, and/or in parallel, and Step Functions will maintain state during executions for you.

5.4 Container image support

AWS Lambda now enables you to package and deploy functions as **container images**. Customers can leverage the flexibility and familiarity of container tooling, and the agility and operational simplicity of AWS Lambda to build applications.

You can start with either an AWS provided base images for Lambda or by using one of your preferred community or private enterprise images. Then use Docker CLI to build the image and upload it to Amazon ECR.

You can deploy third-party Linux base images to Lambda in addition to the Lambda provided images. AWS Lambda will **support all images** based on the following image manifest formats: **Docker Image Manifest V2 Schema 2** or **Open Container Initiative** (OCI).
Lambda supports images with a size of **up to 10GB**.

AWS Lambda provides a variety of base images customers can extend, and customers can also use their preferred Linux-based images.

You can use any container tooling as long as it supports one of the supported container image manifest.

All existing AWS Lambda features, with the exception of Lambda layers and Code Signing, can be used with functions deployed as container images. Once deployed, AWS Lambda will treat an **image as immutable**. Customers can use container layers during their build process to include dependencies.

Your image, once deployed to AWS Lambda, will be immutable. The service will not patch or update the image. However, AWS Lambda will publish curated base images for all supported runtimes that are based on the Lambda managed environment. These published images will be patched and updated along with updates to the AWS Lambda managed runtimes.
This allows you to build and test the updated images and runtimes, prior to deploying the image to production.

There are three main differences between functions created using ZIP archives vs. container images:

- Functions created using ZIP archives have a maximum code package size of 250 MB unzipped, and those created using container images have a maximum image size of 10 GB.

- Lambda uses Amazon ECR as the underlying code storage for functions defined as container images, so a function may not be invocable when the underlying image is deleted from ECR.

- ZIP functions are automatically patched for the latest runtime security and bug fixes. Functions defined as container images are immutable, and customers are responsible for the components packaged in their function.

AWS Lambda ensures that the performance profiles for functions packaged as container images are the same as for those packaged as ZIP archives, including typically sub-second start up times.

There is no additional charge for packaging and deploying functions as container images to AWS Lambda.

The Lambda Runtime Interface Emulator is a proxy for the Lambda Runtime API,which allows customers to locally test their Lambda function packaged as a container image. It is a lightweight web server that converts HTTP requests to JSON events and emulates the Lambda Runtime API. It allows you to locally test your functions using familiar tools such as cURL and the Docker CLI.
You can include the Lambda Runtime Interface Emulator in your container image to have it accept HTTP requests natively instead of the JSON events required for deployment to Lambda.

The Lambda Runtime API in the running Lambda service accepts JSON events and returns responses. The Lambda Runtime Interface Emulator allows the function packaged as a container image to accept HTTP requests during local testing with tools like cURL, and surface them via the same interface locally to the function.

5.5 Provisioned Concurency

Provisioned Concurrency gives you greater control over the performance of your serverless applications. When enabled, Provisioned Concurrency keeps functions initialized and hyper-ready to respond in **double-digit milliseconds**.

The simplest way to benefit from Provisioned Concurrency is by using **AWS Auto Scaling**. You can use Application Auto Scaling to configure schedules, or have Auto Scaling automatically adjust the level of Provisioned Concurrency in real time as demand changes.

You don't need to make any changes to your code to use Provisioned Concurrency. It works seamlessly with all existing functions and runtimes.

Provisioned Concurrency adds a pricing dimension, of **Provisioned Concurrency**, for keeping functions initialized. When enabled, you pay for the amount of concurrency that you configure and for the period of time that you configure it. When your function executes while Provisioned Concurrency is configured on it, you also pay for Requests and execution Duration.

Provisioned Concurrency is ideal for building latency-sensitive applications. You can easily configure the appropriate amount of concurrency based on your application's unique demand. You can increase the amount of concurrency during times of high demand and lower it, or turn it off completely, when demand decreases.

If the concurrency of a function reaches the configured level, subsequent invocations of the function have the latency and scale characteristics of regular Lambda functions. You can restrict your function to only scale up to the configured level. Doing so prevents the function from exceeding the configured level of Provisioned Concurrency.
This is a mechanism to prevent undesired variability in your application when demand exceeds the anticipated amount.

5.6 AWS Lambda functions powered by Graviton2 processors

AWS Lambda allows you to run your functions on either x86-based or Arm-based processors. AWS Graviton2 processors are custom built by Amazon Web Services using 64-bit Arm Neoverse cores to deliver increased price performance for your cloud workloads.
s. Customers get the same advantages of AWS Lambda, running code without provisioning or managing servers, automatic scaling, high availability, and only paying for the resources you consume.

AWS Lambda functions powered by Graviton2, using an Arm-based processor architecture designed by AWS, are designed to deliver up to 34% better price performance compared to functions running on x86 processors, for a variety of serverless workloads.
With lower latency, up to 19% better performance, a 20% lower cost, and the highest power-efficiency currently available at AWS, Graviton2 functions can power mission critical serverless applications.
Customers can configure both existing and new functions to target the Graviton2 processor. They can deploy functions running on Graviton2 as either zip files or container images.

You can configure functions to run on Graviton2 by setting the architecture flag to 'arm64' for your function.

There is no change between x86-based and Arm-based functions. AWS Lambda automatically runs your code when triggered, **without requiring you to provision or manage infrastructure**.

Interpreted languages like Python, Java, and Node generally do not require recompilation unless your code references libraries that use architecture specific components. In those cases, you would need to provide the libraries targeted to arm64.

An application can contain functions running on both architectures. AWS Lambda allows you to change the architecture of your function's current version. **Once you create a specific version of your function, the architecture cannot be changed**.

Each function version can only use a single container image.
Layers and extensions can be targeted to **x86_64** or **arm64** compatible architectures.

At launch, customers can use Python, Node.js, Java, Ruby, .Net Core, Custom Runtime (provided.al2), and OCI Base images.

AWS Lambda functions powered by AWS Graviton2 processors are 20% cheaper compared to x86-based Lambda functions. The Lambda free tier applies to AWS Lambda functions powered by x86 and Arm-based architectures.

5.7 Amazon EFS for AWS Lambda

With (Amazon EFS for AWS Lambda, customers can securely read, write and persist large volumes of data at virtually any scale using a **fully managed elastic NFS file system** that can scale on demand without the need for provisioning or capacity management.
With EFS for Lambda, developers don't need to write code to download data to temporary storage in order to process it.

Developers can easily connect an existing EFS file system to a Lambda function via an **EFS Access Point**
When the function is first invoked, the file system is automatically mounted and made available to function code.

Mount targets for Amazon EFS are associated with a subnet in a VPC. The AWS Lambda function needs to be configured to access that VPC.

Using EFS for Lambda is ideal for building machine learning applications or loading large reference files or models, processing or backing up large amounts of data.
Data encryption in transit uses industry-standard Transport Layer Security (TLS) 1.2 to encrypt data sent between AWS Lambda functions and the Amazon EFS file systems.

Customers can provision Amazon EFS to encrypt data at rest. Data

encrypted at rest is transparently encrypted while being written, and transparently decrypted while being read, so you don't have to modify your applications. Encryption keys are managed by the AWS KMS.

There is no additional charge for using Amazon EFS for AWS Lambda. Customers pay the standard price for AWS Lambda and for Amazon EFS. When using Lambda and EFS in the same availability zone, customers are not charged for data transfer.

Each Lambda function will be able to access one EFS file system.

Amazon EFS supports Lambda functions, ECS and Fargate containers, and EC2 instances. You can share the same file system and use IAM policy and Access Points to control what each function, container, or instance has access to.

5.8 Scalability and availability

AWS Lambda is designed to use replication and redundancy to provide high availability for both the service itself and for the Lambda functions it operates. There are no maintenance windows or scheduled downtimes for either.

When you update a Lambda function, there will be a brief window of time, typically less than a minute, when requests could be served by either the old or the new version of your function.

AWS Lambda is designed to run many instances of your functions in parallel. However, AWS Lambda has a default safety throttle, for the number of concurrent executions per account per region.
You can also control the maximum concurrent executions for individual AWS Lambda functions, which you can use to reserve a subset of your account concurrency limit for critical functions, or cap traffic rates to downstream resources.

On exceeding the maximum concurrent executions limit, AWS Lambda functions being invoked synchronously will return a throttling error (429 error code). Lambda functions being invoked asynchronously can absorb reason-

able bursts of traffic for approximately 15-30 minutes, after which incoming events will be rejected as throttled.

The default maximum concurrent execution limit is applied at the account level. However, you can also set limits on individually functions as well.

Each synchronously invoked Lambda function can scale at a rate of up to 1000 concurrent executions every 10 seconds. While Lambda's scaling rate is suitable for most use cases, it is particularly ideal for those with predictable or unpredictable bursts of traffic.
Each function in your account scales independently of other functions.

On failure, Lambda functions being invoked synchronously will respond with an exception. Lambda functions being invoked asynchronously are retried at least 3 times.

On exceeding the retry policy for asynchronous invocations, you can configure a DLQ into which the event will be placed; in the absence of a configured DLQ the event may be rejected.
You can configure an Amazon SQS queue or an Amazon SNS topic as your dead letter queue.

5.9 Security and access control

You grant permissions to your Lambda function to access other resources using an IAM role. AWS Lambda assumes the role while executing your Lambda function, so you always retain full, secure control of exactly which AWS resources it can use.

When you configure an Amazon S3 bucket to send messages to an AWS Lambda function, a resource policy rule will be created that grants access.

Access controls are managed through the Lambda function role. The role you assign to your Lambda function also determines which resources AWS Lambda can poll on its behalf.

You can enable Lambda functions to access resources in your VPC by specifying the subnet and security group as part of your function configuration. Lambda functions configured to access resources in a particular VPC will not have access to the internet as a default configuration. To grant internet to these functions, use internet gateways.

Code Signing for AWS Lambda offers trust and integrity controls that enable you to verify that only unaltered code from approved developers is deployed in your Lambda functions.
Code Signing for AWS Lambda is currently only available for functions packaged as ZIP archives.
You can enable code signing for existing functions by attaching a code signing configuration to the function.

There is no additional cost when using Code Signing for AWS Lambda. You pay the standard price for AWS Lambda.

5.10 Advanced logging controls

To provide you a simplified and enhanced logging experience by default, AWS Lambda offers advanced logging controls such as the ability to natively capture Lambda function logs in JSON structured format, control the log level filtering of Lambda function logs without making any code changes, and customize the Amazon CloudWatch log group Lambda sends logs to.

You can capture Lambda function logs in JSON structured format without having to use your own logging libraries. JSON structured logs make it easier to search, filter, and analyze large volumes of log entries.

you can use your own logging libraries to generate Lambda logs in JSON structured format. To ensure your logging libraries work seamlessly with Lambda's native JSON structured logging capability, Lambda will not double-encode any logs generated by your function that are already JSON encoded.

There is no additional charge for using advanced logging controls on Lambda. You will continue to be charged for ingestion and storage of your Lambda logs by Amazon CloudWatch Logs.
sec

5.11 Other topics

AWS Lambda is integrated with AWS CloudTrail. AWS CloudTrail can record and deliver log files to your Amazon S3 bucket describing the API usage of your account.

AWS Lambda Envelope encryption file should be stored and referenced as file within the lambda code

AWS Lamabda aliases can be configured to send only a % of traffic to various lambda versions
eg: 20% to v1.0 abd 80% to v2.0

AWS Lambda CPU it's linked to the total ammount of memory RAM it's used/assigned

Lambda maximum size with custom image is **10GiB**

AWS Lambda envelope encryption is used for file larger than 4KiB

6 ASG: Autoscaling Group

Amazon EC2 Auto Scaling helps you ensure that you have the correct number of Amazon EC2 instances available to handle the load for your application. You create collections of EC2 instances, called Auto Scaling groups. You can specify the minimum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures that your group never goes below this size.
You can specify the maximum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures ùthat your group never goes above this size. If you specify the desired capacity, either when you create the group or at any time thereafter, Amazon EC2 Auto Scaling ensures that your group has this many instances.
If you specify scaling policies, then Amazon EC2 Auto Scaling can launch or terminate instances as demand on your application increases or decreases.

6.1 Features of Amazon EC2 Auto Scaling

With Amazon EC2 Auto Scaling, your EC2 instances are organized into Auto Scaling groups so that they can be treated as a logical unit for the purposes of scaling and management. Auto Scaling groups use launch templates as configuration templates for their EC2 instances.

The following are key features of Amazon EC2 Auto Scaling:

- **Monitoring the health of running instances**
  Amazon EC2 Auto Scaling automatically monitors the health and availability of your instances using EC2 health checks and replaces terminated or impaired instances to maintain your desired capacity.

- **Custom health checks**
  You can define custom health checks that are specific to your application to verify that it's responding as expected. If an instance fails your custom health check, it's automatically replaced to maintain your desired capacity.

- **Balancing capacity across Availability Zones**
  You can specify multiple Availability Zones for your Auto Scaling group, and Amazon EC2 Auto Scaling balances your instances evenly across the Availability Zones as the group scales. This provides high availability and resiliency by protecting your applications from failures in a single location.

- **Multiple instance types and purchase options**
  Within a single Auto Scaling group, you can launch multiple instance types and purchase options, allowing you to optimize costs through Spot Instance usage.

- **Automated replacement of Spot Instances**
  If your group includes Spot Instances, Amazon EC2 Auto Scaling can automatically request replacement Spot capacity if your Spot Instances are interrupted. Through Capacity Rebalancing, Amazon EC2 Auto Scaling can also monitor and proactively replace your Spot Instances that are at an elevated risk of interruption.

- **Load balancing**
  You can use Elastic Load Balancing load balancing and health checks

to ensure an even distribution of application traffic to your healthy instances. Whenever instances are launched or terminated, Amazon EC2 Auto Scaling automatically registers and deregisters the instances from the load balancer.

- **Scalability**
  Amazon EC2 Auto Scaling also provides several ways for you to scale your Auto Scaling groups. Using auto scaling allows you to maintain application availability and reduce costs by adding capacity to handle peak loads and removing capacity when demand is lower. You can also manually adjust the size of your Auto Scaling group as needed.

- **Instance refresh**
  The instance refresh feature provides a mechanism to update instances in a rolling fashion when you update your AMI or launch template. You can also use a phased approach, known as a canary deployment, to test a new AMI or launch template on a small set of instances before rolling it out to the whole group.

- **Lifecycle hooks**
  Lifecycle hooks are useful for defining custom actions that are invoked as new instances launch or before instances are terminated. This feature is particularly useful for building event-driven architectures, but it also helps you manage instances through their lifecycle.

- **Support for stateful workloads**
  Lifecycle hooks also offer a mechanism for persisting state on shut down. To ensure continuity for stateful applications, you can also use scale-in protection or custom termination policies to prevent instances with long-running processes from terminating early.

6.1.1 Instance distribution

Amazon EC2 Auto Scaling automatically tries to maintain equivalent numbers of instances in each enabled Availability Zone. Amazon EC2 Auto Scaling does this by attempting to launch new instances in the Availability Zone with the fewest instances.
If there are multiple subnets chosen for the Availability Zone, Amazon EC2 Auto Scaling selects a subnet from the Availability Zone at random. If the

attempt fails, however, Amazon EC2 Auto Scaling attempts to launch the instances in another Availability Zone until it succeeds.

In circumstances where an Availability Zone becomes unhealthy or unavailable, the distribution of instances might become unevenly distributed across the Availability Zones. When the Availability Zone recovers, Amazon EC2 Auto Scaling automatically rebalances the Auto Scaling group.

6.1.2 Rebalancing activities

Rebalancing activities fall into two categories: Availability Zone rebalancing and capacity rebalancing.

- **Availability Zone rebalancing**
  After certain actions occur, your Auto Scaling group can become unbalanced between Availability Zones. Amazon EC2 Auto Scaling compensates by rebalancing the Availability Zones.

  When rebalancing, Amazon EC2 Auto Scaling launches new instances before terminating the earlier ones. This way, rebalancing does not compromise the performance or availability of your application.

  Because Amazon EC2 Auto Scaling attempts to launch new instances before terminating the earlier ones, being at or near the specified maximum capacity could impede or completely halt rebalancing activities.

  the system can temporarily exceed the specified maximum capacity of a group during a rebalancing activity. By default, it can do so by a margin of 10 percent or one instance, whichever is greater. The margin is extended only if the group is at or near maximum capacity and needs rebalancing. The extension lasts only as long as needed to rebalance the group

  You can establish thresholds for an Auto Scaling group by using an instance maintenance policy, and the group can only increase or decrease capacity within that threshold range.

- **Capacity Rebalancing** You can turn on Capacity Rebalancing for your Auto Scaling groups when using Spot Instances. This lets Amazon EC2 Auto Scaling attempt to launch a Spot Instance whenever Amazon EC2 reports that a Spot Instance is at an elevated risk of interruption. After launching a new instance, it then terminates an earlier instance.

6.2 Amazon EC2 Auto Scaling instance lifecycle

The EC2 instances in an Auto Scaling group have a path, or lifecycle, that differs from that of other EC2 instances. The lifecycle starts when the Auto Scaling group launches an instance and puts it into service. The lifecycle ends when you terminate the instance, or the Auto Scaling group takes the instance out of service and terminates it.

6.2.1 Scale out

The following scale-out events direct the Auto Scaling group to launch EC2 instances and attach them to the group:

- You manually increase the size of the group.
- You create a scaling policy to automatically increase the size of the group based on a specified increase in demand.
- You set up scaling by schedule to increase the size of the group at a specific time.

When a scale-out event occurs, the Auto Scaling group launches the required number of EC2 instances, using its assigned launch template. These instances start in the **Pending** state. If you add a lifecycle hook to your Auto Scaling group, you can perform a custom action here.

When each instance is fully configured and passes the Amazon EC2 health checks, it is attached to the Auto Scaling group and it enters the **InService** state. The instance is counted against the desired capacity of the Auto Scaling group.

If your Auto Scaling group is configured to receive traffic from an Elastic Load Balancing load balancer, Amazon EC2 Auto Scaling automatically

registers your instance before it marks the instance as **InService**.

6.2.2 Instances in service

Instances remain in the **InService** state until one of the following occurs:

- A scale-in event occurs, and Amazon EC2 Auto Scaling chooses to terminate this instance in order to reduce the size of the Auto Scaling group.
- You put the instance into a **Standby** state.
- You detach the instance from the Auto Scaling group.
- The instance fails a required number of health checks, so it is removed from the Auto Scaling group, terminated, and replaced.

6.2.3 Scale in

The following scale-in events direct the Auto Scaling group to detach EC2 instances from the group and terminate them:

- The following scale-in events direct the Auto Scaling group to detach EC2 instances from the group and terminate them:
- You create a scaling policy to automatically decrease the size of the group based on a specified decrease in demand.
- You detach the instance from the Auto Scaling group.
- You set up scaling by schedule to decrease the size of the group at a specific time.

It is important that you create a corresponding scale-in event for each scale-out event that you create. This helps ensure that the resources assigned to your application match the demand for those resources as closely as possible.

When a scale-in event occurs, the Auto Scaling group terminates one or more instances. The Auto Scaling group uses its termination policy to determine which instances to terminate. Instances that are in the process of

terminating from the Auto Scaling group enter the **Terminating** state, and can't be put back into service.

If your Auto Scaling group is configured to receive traffic from an Elastic Load Balancing load balancer, Amazon EC2 Auto Scaling automatically deregisters the terminating instance from the load balancer.
If you add a lifecycle hook to your Auto Scaling group, you can perform a custom action on the terminating instance.

6.2.4 Detach an instance

You can detach an instance from your Auto Scaling group. After the instance is detached, you can manage it separately from the Auto Scaling group or attach it to a different Auto Scaling group.

6.2.5 Attach an instance

You can attach a running EC2 instance that meets certain criteria to your Auto Scaling group. After the instance is attached, it is managed as part of the Auto Scaling group.

6.2.6 Enter and exit standby

You can put any instance that is in an **InService** state into a **Standby** state. This enables you to remove the instance from service, troubleshoot or make changes to it, and then put it back into service.
Instances in a **Standby** state continue to be managed by the Auto Scaling group. However, they are not an active part of your application until you put them back into service.

6.3 Launch configurations

A launch configuration is an instance configuration template that an Auto Scaling group uses to launch EC2 instances. When you create a launch configuration, you specify information for the instances. Include the ID of the Amazon Machine Image (AMI), the instance type, a key pair, one or more security groups, and a block device mapping.

You can specify your launch configuration with multiple Auto Scaling groups. However, you can only specify one launch configuration for an Auto Scaling group at a time, and you can't modify a launch configuration after you've created it.
To change the launch configuration for an Auto Scaling group, you must create a launch configuration and then update your Auto Scaling group with it.

6.4 Target tracking scaling policies for Amazon EC2 Auto Scaling

A target tracking scaling policy automatically scales the capacity of your Auto Scaling group based on a target metric value. This allows your application to maintain optimal performance and cost efficiency without manual intervention.

You select a metric and a target value to represent the ideal average utilization or throughput level for your application. Amazon EC2 Auto Scaling creates and manages the CloudWatch alarms that invoke scaling events when the metric deviates from the target.

6.4.1 Multiple target tracking scaling policies

To help optimize scaling performance, you can use multiple target tracking scaling policies together, provided that each of them uses a different metric. Whenever one of these metrics changes, it usually implies that other metrics will also be impacted. The use of multiple metrics therefore provides additional information about the load that your Auto Scaling group is under.

6.4.2 Choose metrics

You can create target tracking scaling policies with either predefined metrics or custom metrics.
You can choose other available CloudWatch metrics or your own metrics in CloudWatch by specifying a custom metric. You must use the AWS CLI or an

SDK to create a target tracking policy with a customized metric specification.

Keep the following in mind when choosing a metric:

AWS Autoscaling can not increase the number of instances over the max ceiling set on the ASG

- **ASGAverageCPUUtilization**: Average CPU utilization of the Auto Scaling group.
- **ASGAverageNetworkIn**:Average number of bytes received by a single instance on all network interfaces.
- **ASGAverageNetworkOut**: Average number of bytes sent out from a single instance on all network interfaces.
- **ALBRequestCountPerTarget**: Average Application Load Balancer request count per target.

You can choose other available CloudWatch metrics or your own metrics in CloudWatch by specifying a custom metric.

Keep the following in mind when choosing a metric:

- We recommend that you only use metrics that are available at one-minute intervals to help you scale faster in response to utilization changes. Target tracking will evaluate metrics aggregated at a one-minute granularity for all predefined metrics and custom metrics, but the underlying metric might publish data less frequently.
- Not all custom metrics work for target tracking. The metric must be a valid utilization metric and describe how busy an instance is. The metric value must increase or decrease proportionally to the number of instances in the Auto Scaling group. That's so the metric data can be used to proportionally scale out or in the number of instances.
- To use the **ALBRequestCountPerTarget** metric, you must specify the ResourceLabel parameter to identify the load balancer target group that is associated with the metric.

- When a metric emits real 0 values to CloudWatch, an Auto Scaling group can scale in to 0 when there is no traffic to your application for a sustained period of time. To have your Auto Scaling group scale in to 0 when no requests are routed it, the group's minimum capacity must be set to 0.
- Instead of publishing new metrics to use in your scaling policy, you can use metric math to combine existing metrics.

6.4.3 Define target value

When you create a target tracking scaling policy, you must specify a target value. The target value represents the optimal average utilization or throughput for the Auto Scaling group. To use resources cost efficiently, set the target value as high as possible with a reasonable buffer for unexpected traffic increases. When your application is optimally scaled out for a normal traffic flow, the actual metric value should be at or just below the target value.

When a scaling policy is based on throughput the target value represents the optimal average throughput from a single instance, for a **one-minute period**.

6.4.4 Define instance warmup time

You can optionally specify the number of seconds that it takes for a newly launched instance to warm up. Until its specified warmup time has expired, an instance is not counted toward the aggregated EC2 instance metrics of the Auto Scaling group.

While instances are in the warmup period, your scaling policies only scale out if the metric value from instances that are not warming up is greater than the policy's target utilization.
If the group scales out again, the instances that are still warming up are counted as part of the desired capacity for the next scale-out activity. The intention is to **continuously scale out**.
While the scale-out activity is in progress, all scale-in activities initiated by scaling policies are blocked until the instances finish warming up. When the instances finish warming up, if a scale-in event occurs, any instances currently

in the process of terminating will be counted towards the current capacity of the group when calculating the new desired capacity. We **don't remove more instances from the Auto Scaling group than necessary**.
If no value is set, then the scaling policy will use the default value, which is the value for the default instance warmup defined for the group. If the default instance warmup is null, then it falls back to the value of the default cooldown.

6.5 Monitor your Auto Scaling groups

Monitoring is an important part of maintaining the reliability, availability, and performance of Amazon EC2 Auto Scaling and your AWS Cloud solutions. AWS provides the following tools to watch Amazon EC2 Auto Scaling, report when something is wrong, and take automatic actions when appropriate.

6.6 Health checks for instances in an Auto Scaling group

Amazon EC2 Auto Scaling continuously monitors the health status of instances in an Auto Scaling group to maintain the desired capacity.

All instances in an Auto Scaling group start with a **Healthy** status. Instances are assumed to be healthy unless Amazon EC2 Auto Scaling receives notification that they are unhealthy. It can receive notifications from various sources when an instance becomes unhealthy and must be replaced.

When Amazon EC2 Auto Scaling determines that an **InService** instance is unhealthy, it replaces it with a new instance to maintain the desired capacity of the group. The new instance launches using the current settings of the Auto Scaling group and its associated launch template or launch configuration.

Unhealthy instances can also occur when an instance terminates unexpectedly, such as from a Spot Instance interruption or manual termination by a user.

6.7 Infrastructure security in Amazon EC2 Auto Scaling

As a managed service, Amazon EC2 Auto Scaling is protected by AWS global network security.
You use AWS published API calls to access Amazon EC2 Auto Scaling through the network. Clients must support the following:

- Transport Layer Security (TLS). We require TLS 1.2 and recommend TLS 1.3.
- Cipher suites with perfect forward secrecy (PFS) such as DHE (Ephemeral Diffie-Hellman) or ECDHE (Elliptic Curve Ephemeral Diffie-Hellman).

equests must be signed by using an access key ID and a secret access key that is associated with an IAM principal. Or you can use AWS STS to generate temporary security credentials to sign requests.

You can also use a virtual private cloud (VPC) endpoint for Amazon EC2 Auto Scaling. Interface VPC endpoints enable your Amazon VPC resources to use their private IP addresses to access Amazon EC2 Auto Scaling with no exposure to the public internet.

7 Amazon Elastic Block Store

Amazon Elastic Block Store (Amazon EBS) provides scalable, high-performance block storage resources that can be used with Amazon EC2 instances. With Amazon Elastic Block Store, you can create and manage the following block storage resources:

- **Amazon EBS volumes**
  These are storage volumes that you attach to Amazon EC2 instances. After you attach a volume to an instance, you can use it in the same way you would use a local hard drive attached to a computer
- **Amazon EBS snapshots**
  These are point-in-time backups of Amazon EBS volumes that persist independently from the volume itself. You can create snapshots to back

up the data on your Amazon EBS volumes. You can then restore new volumes from those snapshots at any time.

Amazon EBS provides the following features and benefits:

- **Multiple volume types**
  Amazon EBS provides multiple volume types that allow you to optimize storage performance and cost for a broad range of applications. Volume types are divided into two major categories: SSD-backed storage for transactional workloads, and HDD-backed storage for throughput intensive workloads.

- **Scalability**
  You can create Amazon EBS volumes with capacity and performance specifications that meet your needs. As your needs changes, you can use Elastic Volumes operations to dynamically increase capacity or tune performance, with no downtime.

- **Backup and recovery**
  Use Amazon EBS snapshots to back up the data stored on your volumes. You can then use those snapshots to instantly restore volumes or to migrate data across AWS accounts, AWS Regions, or Availability Zones.

- **Data protection**
  Use Amazon EBS encryption to encrypt your Amazon EBS volumes and Amazon EBS snapshots. Encryption operations occur on the servers that host Amazon EC2 instances, ensuring the security of both data-at-rest and data-in-transit between an instance and its attached volume and subsequent snapshots.

- **Data availability and durability**
  io2 Block Express volumes provide 99.999% durability with an annual failure rate of 0.001%. Other volume types provide 99.8% to 99.9% durability with an annual failure rate of 0.1% to 0.2%. Additionally, volume data is automatically replicated across multiple servers in an Availability Zone to prevent the loss of data from the failure of any single component.

- **Data archiving**
  EBS Snapshots Archive provides a low-cost storage tier to archive full, point-in-time copies of EBS Snapshots that you must retain for 90 days or more for regulatory and compliance reasons, or for future project releases.

7.1 Amazon EBS volumes

An Amazon EBS volume is a durable, block-level storage device that you can attach to your instances. After you attach a volume to an instance, you can use it as you would use a physical hard drive. EBS volumes are flexible. For current-generation volumes attached to current-generation instance types, you can dynamically increase size, modify the provisioned IOPS capacity, and change volume type on live production volumes.

You can use EBS volumes as primary storage for data that requires frequent updates. You can also use them for throughput-intensive applications that perform continuous disk scans. EBS volumes persist independently from the running life of an EC2 instance.

You can attach multiple EBS volumes to a single instance. The volume and instance must be in the same Availability Zone. Depending on the volume and instance types, you can use Multi-Attach to mount a volume to multiple instances at the same time.

EBS volumes provide benefits that are not provided by instance store volumes.

7.1.1 Data availability

When you create an EBS volume, it is automatically replicated within its Availability Zone to prevent data loss due to failure of any single hardware component. You can attach an EBS volume to any EC2 instance in the same Availability Zone. After you attach a volume, it appears as a native block device similar to a hard drive or other physical device. You can connect to the instance and format the EBS volume with a file system, such as **Ext4** for a Linux instance or **NTFS** for a Windows instance, and then install applications.

If you attach multiple volumes to a device that you have named, you can stripe data across the volumes for increased I/O and throughput performance.
You can get monitoring data for your EBS volumes, including root device volumes for EBS-backed instances, at no additional charge.

7.1.2 Data persistence

An EBS volume is off-instance storage that can persist independently from the life of an instance. You continue to pay for the volume usage as long as the data persists.

EBS volumes that are attached to a running instance can automatically detach from the instance with their data intact when the instance is terminated if you uncheck the **Delete on Termination** check box when you configure EBS volumes
The volume can then be reattached to a new instance, enabling quick recovery. If the check box for **Delete on Termination** is checked, the volume(s) will delete upon termination of the EC2 instance.
If you are using an EBS-backed instance, you can stop and restart that instance without affecting the data stored in the attached volume. The volume remains attached throughout the stop-start cycle. This enables you to process and store the data on your volume indefinitely, only using the processing and storage resources when required. The data persists on the volume until the volume is deleted explicitly.
If you are dealing with sensitive data, you should consider encrypting your data manually or storing the data on a volume protected by Amazon EBS encryption.

the root EBS volume that is created and attached to an instance at launch is deleted when that instance is terminated. You can modify this behavior by changing the value of the flag **DeleteOnTermination** to **false** when you launch the instance.
additional EBS volumes that are created and attached to an instance at launch are not deleted when that instance is terminated.

7.1.3 Data encryption

For simplified data encryption, you can create encrypted EBS volumes with the Amazon EBS encryption feature. All EBS volume types support encryption. You can use encrypted EBS volumes to meet a wide range of data-at-rest encryption requirements for regulated/audited data and applications. Amazon EBS encryption uses 256-bit AES-256 and an Amazon-managed key infrastructure. The encryption occurs on the server that hosts the EC2 instance, providing encryption of data-in-transit from the EC2 instance to Amazon EBS storage.

Amazon EBS encryption uses AWS KMS keys when creating encrypted volumes and any snapshots created from your encrypted volumes. The first time you create an encrypted EBS volume in a Region, a default AWS managed KMS key is created for you automatically. This key is used for Amazon EBS encryption unless you create and use a customer managed key.

7.1.4 Data security

Amazon EBS volumes are presented to you as raw, unformatted block devices. These devices are logical devices that are created on the EBS infrastructure and the Amazon EBS service ensures that the devices are logically empty prior to any use or re-use by a customer.
If you have procedures that require that all data be erased using a specific method, you have the ability to do so on Amazon EBS.

7.1.5 Snapshots

Amazon EBS provides the ability to create snapshots (backups) of any EBS volume and write a copy of the data in the volume to Amazon S3, where it is stored redundantly in multiple Availability Zones. The volume does not need to be attached to a running instance in order to take a snapshot. As you continue to write data to a volume, you can periodically create a snapshot of the volume to use as a baseline for new volumes. These snapshots can be used to create multiple new EBS volumes or move volumes across Availability Zones. Snapshots of encrypted EBS volumes are automatically encrypted.

When you create a new volume from a snapshot, it's an exact copy of the original volume at the time the snapshot was taken. EBS volumes that are created from encrypted snapshots are automatically encrypted. By optionally specifying a different Availability Zone, you can use this functionality to create a duplicate volume in that zone. The snapshots can be shared with specific AWS accounts or made public. When you create snapshots, you incur charges in Amazon S3 based on the size of the data being backed up, not the size of the source volume. Subsequent snapshots of the same volume are incremental snapshots. They include only changed and new data written to the volume since the last snapshot was created.

Snapshots are incremental backups, meaning that only the blocks on the volume that have changed after your most recent snapshot are saved.
Even though snapshots are saved incrementally, the snapshot deletion process is designed so that you need to retain only the most recent snapshot.

7.1.6 Flexibility

EBS volumes support live configuration changes while in production. You can modify volume type, volume size, and IOPS capacity without service interruptions.

7.2 Amazon EBS volume types

Amazon EBS provides the following volume types, which differ in performance characteristics and price, so that you can tailor your storage performance and cost to the needs of your applications.

The volume type that AWS EBS support are the following:

- Solid state drive (SSD) volumes
- Hard disk drive (HDD) volumes
- Previous generation volumes

7.2.1 Solid state drive (SSD) volumes

SSD-backed volumes are optimized for transactional workloads involving frequent read/write operations with small I/O size, where the dominant performance attribute is IOPS. SSD-backed volume types include **General Purpose SSD** and **Provisioned IOPS SSD**.
SSD Volumes support **gp3**, **gp2**, **io2** and **io1** volume type.

7.2.2 Hard disk drive (HDD) volumes

HDD-backed volumes are optimized for large streaming workloads where the dominant performance attribute is throughput. HDD volume types include **Throughput Optimized HDD** and **Cold HDD**. HDD volume support **st1** and **sc1** volume type.

7.2.3 Previous generation volumes

Magnetic (**standard**) volumes are previous generation volumes that are backed by magnetic drives. They are suited for workloads with small datasets where data is accessed infrequently and performance is not of primary importance.

7.3 Constraints on the size and configuration of an EBS volume

The size of an Amazon EBS volume is constrained by the physics and arithmetic of block data storage, as well as by the implementation decisions of operating system (OS) and file system designers. AWS imposes additional limits on volume size to safeguard the reliability of its services.

Amazon EBS abstracts the massively distributed storage of a data center into virtual hard disk drives. To an operating system installed on an EC2 instance, an attached EBS volume appears to be a physical hard disk drive containing 512-byte disk sectors. The OS manages the allocation of data blocks (or clusters) onto those virtual sectors through its storage management utilities. The allocation is in conformity with a volume partitioning scheme, such as master boot record (MBR) or GUID partition table (GPT), and within the capabilities of the installed file system (ext4, NTFS, and so

on).

Among other impacts, the partitioning scheme determines how many logical data blocks can be uniquely addressed in a single volume.
The common partitioning schemes in use are Master Boot Record (MBR) and GUID partition table (GPT).

7.3.1 Data block sizes

Data storage on a modern hard drive is managed through logical block addressing, an abstraction layer that allows the operating system to read and write data in logical blocks without knowing much about the underlying hardware. The OS relies on the storage device to map the blocks to its physical sectors. EBS advertises 512-byte sectors to the operating system, which reads and writes data to disk using data blocks that are a multiple of the sector size.
The industry default size for logical data blocks is currently 4,096 bytes (4 KiB). Because certain workloads benefit from a smaller or larger block size, file systems support non-default block sizes that can be specified during formatting.

7.4 Amazon EBS volume lifecycle

The lifecycle of an Amazon EBS volume starts with the creation process. You can create a volume from an Amazon EBS snapshot or you can create an empty volume. Before you can use your volume, you must attach it to one or more Amazon EC2 instances that are in the same Availability Zone as the volume. You can attach multiple volumes to an instance. If needed, you can detach a volume from one instance and then attach it to another instance. If your storage requirements change, you can modify the size or performance of the volume at any time.
If you no longer need a volume, you can delete it to stop incurring the related storage costs.

You can attach an available EBS volume to one or more of your instances that is in the same Availability Zone as the volume.

7.5 Amazon EBS snapshots

You can back up the data on your Amazon EBS volumes by making point-in-time copies, known as Amazon EBS snapshots. A snapshot is an **incremental backup**, which means that we save only the blocks on the device that have changed since your most recent snapshot.

EBS snapshots are stored in Amazon S3, in S3 buckets that you can't access directly. You can't access your snapshots using the Amazon S3 console or the Amazon S3 API.
Each snapshot contains all of the information that is needed to restore your data to a new EBS volume. When you create an EBS volume based on a snapshot, the new volume begins as an exact replica of the volume that was used to create the snapshot. The replicated volume loads data in the background so that you can begin using it immediately.
When you delete a snapshot, only the data unique to that snapshot is removed.

You can track the status of your EBS snapshots through CloudWatch Events.

Snapshots can be used to create a backup of critical workloads, such as a large database or a file system that spans across multiple EBS volumes. Multi-volume snapshots allow you to take exact point-in-time, data coordinated, and crash-consistent snapshots across multiple EBS volumes attached to an EC2 instance.

Charges for your snapshots are based on the amount of data stored. Because snapshots are incremental, deleting a snapshot might not reduce your data storage costs. Data referenced exclusively by a snapshot is removed when that snapshot is deleted, but data referenced by other snapshots is preserved.

7.5.1 How snapshots work

The first snapshot that you create from a volume is always a full snapshot. It includes all of the data blocks written to the volume at the time of creating the snapshot. Subsequent snapshots of the same volume are incremental snapshots. They include only changed and new data blocks written to the

volume since the last snapshot was created.

The size of a full snapshot is determined by the size of the data being backed up, not the size of the source volume. Similarly, the storage costs associated with a full snapshot is determined by the size of the snapshot, not the size of the source volume.
Similarly, the size and storage costs of an incremental snapshot are determined by the size of any data that was written to the volume since the previous snapshot was created.

7.5.2 Copy and share snapshots

You can share a snapshot across AWS accounts by modifying its access permissions. You can make copies of your own snapshots as well as snapshots that have been shared with you.
A snapshot is constrained to the AWS Region where it was created. After you create a snapshot of an EBS volume, you can use it to create new volumes in the same Region.
. You can also copy snapshots across Regions, making it possible to use multiple Regions for geographical expansion, data center migration, and disaster recovery. You can copy any accessible snapshot that has a **completed** status.

7.6 Amazon EBS encryption

Use Amazon EBS encryption as a straight-forward encryption solution for your EBS resources associated with your EC2 instances. With Amazon EBS encryption, you aren't required to build, maintain, and secure your own key management infrastructure. Amazon EBS encryption uses AWS KMS keys when creating encrypted volumes and snapshots.

Encryption operations occur on the servers that host EC2 instances, ensuring the security of both data-at-rest and data-in-transit between an instance and its attached EBS storage.

You can attach both encrypted and unencrypted volumes to an instance simultaneously.

7.6.1 How EBS encryption works

You can encrypt both the boot and data volumes of an EC2 instance. When you create an encrypted EBS volume and attach it to a supported instance type, the following types of data are encrypted:

- Data at rest inside the volume
- All data moving between the volume and the instance
- All snapshots created from the volume
- All volumes created from those snapshots

Amazon EBS encrypts your volume with a data key using industry-standard AES-256 data encryption. The data key is generated by AWS KMS and then encrypted by AWS KMS with your AWS KMS key prior to being stored with your volume information. All snapshots, and any subsequent volumes created from those snapshots using the same AWS KMS key share the same data key.

7.6.2 How unusable KMS keys affect data keys

When a KMS key becomes unusable, the effect is almost immediate. The key state of the KMS key changes to reflect its new condition, and all requests to use the KMS key in cryptographic operations fail.

When you perform an action that makes the KMS key unusable, there is no immediate effect on the EC2 instance or the attached EBS volumes. Amazon EC2 uses the data key, not the KMS key, to encrypt all disk I/O while the volume is attached to the instance.

7.6.3 Encrypt EBS resources

You encrypt EBS volumes by enabling encryption, either using encryption by default or by enabling encryption when you create a volume that you want

to encrypt.
When you encrypt a volume, you can specify the symmetric encryption KMS key to use to encrypt the volume. If you do not specify a KMS key, the KMS key that is used for encryption depends on the encryption state of the source snapshot and its ownership.

You cannot change the KMS key that is associated with an existing snapshot or volume. However, you can associate a different KMS key during a snapshot copy operation so that the resulting copied snapshot is encrypted by the new KMS key.

You cannot directly encrypt existing unencrypted volumes or snapshots. However, you can create encrypted volumes or snapshots from unencrypted volumes or snapshots. If you enable encryption by default, Amazon EBS automatically encrypts new volumes and snapshots using your default KMS key for EBS encryption. Otherwise, you can enable encryption when you create an individual volume or snapshot, using either the default KMS key for Amazon EBS encryption or a symmetric customer managed encryption key.
To encrypt the snapshot copy to a customer managed key, you must both enable encryption and specify the KMS key, as shown in Copy an unencrypted snapshot (encryption by default not enabled).

Amazon EBS does not support asymmetric encryption KMS keys.

You can also apply new encryption states when launching an instance from an EBS-backed AMI. This is because EBS-backed AMIs include snapshots of EBS volumes that can be encrypted as described.

7.6.4 Rotating AWS KMS keys

Cryptographic best practices discourage extensive reuse of encryption keys. To create new cryptographic material for use with Amazon EBS encryption, you can either create a new customer managed key, and then change your applications to use that new KMS key. Or, you can enable automatic key rotation for an existing customer managed key.

When you enable automatic key rotation for a customer managed key, AWS KMS generates new cryptographic material for the KMS key every year. AWS KMS saves all previous versions of the cryptographic material so that you can continue to decrypt and use volumes and snapshots previously encrypted with that KMS key material. AWS KMS does not delete any rotated key material until you delete the KMS key.

When you use a rotated customer managed key to encrypt a new volume or snapshot, AWS KMS uses the current (new) key material.

7.7 Amazon EBS volume performance

Several factors, including I/O characteristics and the configuration of your instances and volumes, can affect the performance of Amazon EBS.
We recommend that you tune performance with information from your actual workload, in addition to benchmarking, to determine your optimal configuration.
AWS updates to the performance of EBS volume types might not immediately take effect on your existing volumes. To see full performance on an older volume, you might first need to perform a **ModifyVolume** action on it.

7.7.1 IOPS

IOPS are a unit of measure representing input/output operations per second. The operations are measured in KiB, and the underlying drive technology determines the maximum amount of data that a volume type counts as a single I/O.

I/O size is capped at 256 KiB for SSD volumes and 1,024 KiB for HDD volumes because SSD volumes handle small or random I/O much more efficiently than HDD volumes.

When small I/O operations are physically sequential, Amazon EBS attempts to merge them into a single I/O operation up to the maximum I/O size. Similarly, when I/O operations are larger than the maximum I/O size, Amazon EBS attempts to split them into smaller I/O operations.

The volume queue length is the number of pending I/O requests for a device. Latency is the true end-to-end client time of an I/O operation, in other words, the time elapsed between sending an I/O to EBS and receiving an acknowledgement from EBS that the I/O read or write is complete.
Queue length must be correctly calibrated with I/O size and latency to avoid creating bottlenecks either on the guest operating system or on the network link to EBS.
If your workload is not delivering enough I/O requests to fully use the performance available to your EBS volume, then your volume might not deliver the IOPS or throughput that you have provisioned.

7.8 Security in Amazon Elastic Block Store

Cloud security at AWS is the highest priority. As an AWS customer, you benefit from data centers and network architectures that are built to meet the requirements of the most security-sensitive organizations.

7.9 Data protection

For data protection purposes, we recommend that you protect AWS account credentials and set up individual users with AWS IAM Identity Center or AWS Identity and Access Management (IAM). That way, each user is given only the permissions necessary to fulfill their job duties.

7.9.1 Amazon EBS data security

Amazon EBS volumes are presented to you as raw, unformatted block devices. These devices are logical devices that are created on the EBS infrastructure and the Amazon EBS service ensures that the devices are logically empty prior to any use or re-use by a customer.

7.9.2 Encryption at rest and in transit

Amazon EBS encryption is an encryption solution that enables you to encrypt your Amazon EBS volumes and Amazon EBS snapshots using AWS

Key Management Service cryptographic keys. EBS encryption operations occur on the servers that host Amazon EC2 instances, ensuring the security of both data-at-rest and data-in-transit between an instance and its attached volume and any subsequent snapshots.

7.9.3 KMS key management

When you create an encrypted Amazon EBS volume or snapshot, you specify an AWS Key Management Service key. By default, Amazon EBS uses the AWS managed KMS key for Amazon EBS in your account and Region (aws/ebs). However, you can specify a customer managed KMS key that you create and manage. Using a customer managed KMS key gives you more flexibility, including the ability to create, rotate, and disable KMS keys.

To use a customer managed KMS key, you must give users permission to use the KMS key.

- All at once
- Rolling
- Rolling with additional batch
- Immutable

Rolling and Rolling with additional batch deployment policy are not suitable for the single instance environment, because they need a Elastic Load-Balancer (ELB) that attach and detach the instances

7.9.4 All At Once

Deploy the application version to all instances at the same time.
This is the **fastest** but also the most **dangerous** deployment method.
Fast because you are deploy the same app version to all the instances all at once.
Dangerous because all the instances are taken out of services all at once, meaning your service will become unavailable. A majour failure could be catastrophic.

If there is a failuer the roll back strategy will roll back to the latest working version the server all at once.

7.9.5 Rolling

Rolling deployment will depoloy the new version of the application to a **batch** of instances at the same time.
It will take batch of instances out of services while the deployment process is running.
Once the process is complete the updated instances are reattached.
The process is slower that **All At Once** strategy but safer.

7.9.6 Rolling With Additional Batch

Rolling With Additional Batch will lauch new instances that will be used to replace batches of old one.
The old version of the server are terminated once the new one are attacched.

With this stratedy the server capacity is never reduced. This is important for application where redundancy is the key aspect.

7.9.7 Immutable

Immutable deploy rely heavily on the **Auto Scaling Groups** (ASG).
It will create a new ASG with EC2 instances, Deploy the updated version of the app on the new EC2 instances and point the ELB to the new ASG.
The old ASG is terminated as soon as the new one is attacched.
Immutable is the **safest** way to deploy critical application.

7.10 Deployment Methods

8 Developer note

Altough temporary credentials with "get-session-token" are most secure way to obtain extended privilages, it's safer to assume dev account already has enough access to get a job done, than that there is additional role with more privilages that can be fetched via this command.

The developer can pass the log group's Amazon Resource Name (ARN) as an environment variable to the Lambda function to meet this requirement. By specifying the log group's ARN as an environment variable in the CloudFormation template, the developer can make the ARN available to the Lambda function at runtime. The function can then use the ARN to write logs to the specified log group in Amazon CloudWatch Logs.

Canary deployment expose X% of the traffic at the start to enpoint B and then 100% after an initial delay set by the user

If we get the IP directly from the http request, we can notice that it is always the same and it changes rarely. This because the request comes from the ALB. Instead, we can use the **"X-Forwarded-For"** header to get the original IP of the client

Amazon RDS Read Replicas provide enhanced performance and durability for RDS database (DB) instances. They make it easy to elastically scale out beyond the capacity constraints of a single DB instance for read-heavy database workloads.

Amazon ElastiCache for Redis is a caching layer that can be used to improve the performance of data retrieval from a database. It supports encryption at rest and in transit, and can be configured for high availability by using Redis Cluster

The output of a state can be a copy of its input, the result it produces (for example, output from a Task state's Lambda function), or a combination of its input and result. Use ResultPath to control which combination of these is passed to the state output.

To allow the developer to perform a code push efficiently, without the need to update the API Gateway", this will require creating an alias referencing latest lambda version abd which can be referred to using API Gateway Stage Variable.

When you register a task definition, you must specify a list of container definitions that are passed to the Docker daemon on a container instance.

Based of the kind of hw it's possible to differentiate the logic. To retrieve this metadata, the application should launch a curl command to the following link: **http://169.254.169.254/latest/meta-data/**.

9 AWS SAM

AWS SAM applicat can be redeployed with the **build** and **deploy** command **Transform** operation are used on SAM model

10 AWS KMS

AWS encrypt file with AES-256 encryption algorithm
AWs encryption can be used only along with HTTPS

11 AWS CLI

AWS CLI command can be testd with **-dry-run** flag during command option CLI Command can use the **-region** command flag to specify the region where the CLI command is run

12 CloudFront

Amazon CloudFront work on a Content Delivery Network (CDN) that improve read performance for a data a using a cache at edge
AWS CloudFront signed URL can be used to controll the access of file under

the CloudFront Network.
CloudFront private and public key are used to sign a portion of the Cloud-Front URL.
CloudFront key pairs can be created only by root user

You can import private and public SSH key into each other. You can also reuse SSH key Through AWS region

13 CloudFormation

The correct format for the CloudFormation **FindInMap** operation is:

!FindInMap[MapName, TopLevelKey, SecondLevelKey]

There is a **Dependency** section on CloudFormation templates

14 AWS VPN

AWS Subnet allow VPN partitioning
Routing tabkle defines the internet access for a VPN Subnet

NAT gateway are managed by AWS and allow a subnet to access internet while remaining private
NACL can have Allow and Deny rules while Security Groups only have Allow rules and leave everything elese unspecified to deny.

VPC flow logs are a feature that enable to capture info about IP traffic to and from the network

15 AWS EC2

EC2 Metadata are located on **http://<instance-ip>/latest/metadata**
On a 5XX server error with API you should implement exponential backoff

16 AWS SQL

Aurora SQL is the AWS default engine for the RDS databases engine it's proprietrary of AWS (no opensource), it cost 20% more then the most expensive engine on RDS
Replication of instances over the RDS apps uses one instances for write operation and the following for read operation

17 Route 53

Route 53 CNAME are aliases for other host with record tyoe **A** or **AAAA**
Route 53 Aliases are used to point to AWS resources hostnames
Aliases work for root domain and non-root domain
Aliases record map hostnames to AWS resources

18 ElasticBeanstalk

Any resources created inside the ElasticBeanstalk directory **.ebextensions** is part of tge EB template and it wull be deleted after redeploying the template

.ebextension is the directory responable on EB to store user custom resources on AWS

ElasticBeanstalk rolling with additional batches means that rollout is made on capacity with 100% of instances running at the same time
ElasticBeanstalk blue/green deployment involve the Route53 wigthed traffic
Resources managed on the .ebextensions directory are deleted on environment changes

AWS Beanstalk it's used to create and deploy web and mobile application

Immutable deployment is the minimum config that mantain alive the older application on ElasticBeanstalk

19 API Gateway

AWS API Gateway can access an internal cache
AWS API Gateway can leverage AWS Lambda computational ability to run an external authentication on the API Gateway

20 AWS Budget

AWS Budget need a **minimum of 5 week** before it can show metrics extracted from budget and show budget forecast

21 AWS S3

AWS S3 have the capacity to deny PutObject action for object uploaded and not encrypter with **x-amz-server-side-encryption** api header

AWS S3 presigned-URL can be time limited with a min value of 1 minute and a maximum of 12 hours

22 AWS RDS

One ore more read replicas on RDS could help on read-heavy operation

23 AWS CDK

AWS CDK could define AWS resources on different programming languages that suppord the AWS CDK library
Build Option (eg: java build appname) is optional on CDK application for languages that support a build before a run operation

24 AWS IAM

IAM Access Analyzer help you identify resources on your organization that are shared with external entity
IAM user are able, unless specifically denied, to create SSL/TLS certificate

CodeCommit and CodeDeploy use appspec.yaml file on root dit to define build option and configuration.
ValidateService is the last CodeDeploy hook available after an applicatin is installed and it's running